TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Bài tập lớn 2 (cuối kì): từ yêu cầu đến triển khai

BrewLite – Ứng dụng đặt đồ uống

GVHD: Từ Lăng Phiêu
SV: Nguyễn Lê Nhựt Thắng - [MSSV]
Hồ Thiên Ngân - [MSSV]
Hà Thiên Long - [MSSV]
Giản Thiên Lực - [MSSV]

TP. HỒ CHÍ MINH, THÁNG 11/2026

Contents

Danh mục từ viết tắt **5**

Bảng thuật ngữ **6**

Tóm tắt **7**

1 Giới thiệu và quy trình phát triển **8**
1.1 Bối cảnh, mục tiêu, phạm vi 8
1.2 Mô hình quy trình áp dụng 8
1.3 Công nghệ và công cụ sử dụng 8
1.4 Cấu trúc báo cáo 9

2 Quản lý dự án (Scrum) **9**
2.1 Vai trò / trách nhiệm trong nhóm 9
2.2 Sự kiện và artefact 9
2.3 Kế hoạch và diễn biến từng Sprint 9
2.3.1 Sprint 1 (20–23/10/2026) 9
2.3.2 Sprint 2 (24–26/10/2026) 10
2.4 Definition of Done 10

3 Đặc tả yêu cầu **11**
3.1 Bối cảnh, phạm vi (trong / ngoài) 11
3.2 Stakeholder 11
3.3 Giả định, ràng buộc, phụ thuộc 11
3.4 Yêu cầu chức năng 12
3.5 Yêu cầu phi chức năng 13
3.6 Use case và đặc tả use case 14

4 Mô hình hóa (UML) **16**
4.1 Sơ đồ hoạt động (Activity) 16
4.2 Sơ đồ tuần tự (Sequence) 16
4.3 Sơ đồ lớp / mô hình miền (Class / Domain) 19
4.4 Sơ đồ trạng thái (State machine) 19

5 Kiến trúc hệ thống **19**
5.1 Kiến trúc tổng thể 19
5.2 Quyết định kiến trúc (ADR) 26
5.3 Sơ đồ triển khai (Deployment) 27
5.4 Cơ sở dữ liệu và API 29

6 Triển khai và tích hợp liên tục **30**
6.1 Kho mã và quản lý tài liệu 30
6.2 Hiện thực các chức năng chính 30
6.3 CI 30
6.4 CD, di trú dữ liệu và quay lui 37

7 Kiểm thử **37**
7.1 Chiến lược kiểm thử 37
7.2 Test case 41
7.3 Kết quả kiểm thử 42

8 Bàn giao và đánh giá nhìn lại **43**
8.1 Sản phẩm bàn giao 43
8.2 Sprint Review / Demo 43
8.3 Retrospective 44

9 Ma trận truy vết **44**

10 Kết luận **45**
10.1 Kết quả đạt được 45
10.2 Hạn chế 45
10.3 Hướng phát triển 45

A Bảng đối chiếu yêu cầu đề bài với báo cáo **46**

B Tư liệu bổ sung **46**
B.1 Cấu trúc kho mã 46
B.2 Liên kết 46

List of Figures

1 Use case tổng quan của BrewLite . . . 14
2 Use case của phạm vi UC-01 . . . 15
3 Sơ đồ hoạt động: đặt đơn (phần 1/2) . . . 17
4 Sơ đồ hoạt động: thanh toán và xử lý đơn (phần 2/2) . . . 18
5 Sơ đồ tuần tự: đặt đơn (UC-01) . . . 20
6 Sơ đồ tuần tự: thanh toán (UC-01) . . . 21
7 Sơ đồ lớp: đặt hàng và điểm thưởng . . . 22
8 Sơ đồ lớp: thanh toán, phiên QR và quà đổi điểm . . . 23
9 Sơ đồ lớp: kho nguyên liệu và công thức . . . 24
10 Kiến trúc mức container của BrewLite . . . 25
11 Sơ đồ triển khai (deployment) của BrewLite . . . 27
12 Quy trình CI/CD của BrewLite (activity diagram) . . . 28
13 Sơ đồ thực thể – quan hệ theo `schema.prisma` . . . 29
14 Menu tải từ API trên production . . . 30
15 Chọn size, mức đường, mức đá . . . 31
16 Giỏ hàng: 10 ly, 34.000 đồng/ly, tạm tính 340.000 đồng . . . 31
17 Thanh toán bằng thẻ mô phỏng, mã `WELCOME10` . . . 32
18 Đơn #1 PAID, 306.000 đồng (340.000 đồng giảm 10%) . . . 32
19 Giả lập thanh toán thất bại: đơn #2 PAYMENT_FAILED, có nút "Thử thanh toán lại" . . . 33
20 Thử lại thành công: đơn #2 PAID, 585.000 đồng . . . 33
21 Lịch sử có đúng hai đơn #1 và #2, không tạo đơn trùng khi thử lại . . . 34
22 Trình duyệt báo kết nối an toàn (HTTPS) . . . 34
23 `/health` trả `{"status":"ok","service":"brewlite-backend"}` . . . 35
24 Test lỗi khi chạy ở máy: `expected false to be true` . . . 35
25 CI run #13 đỏ: job `api` lỗi, các job còn lại xanh . . . 35
26 PR #30: check `api` (Required) đỏ, "Merging is blocked" . . . 36
27 CI run #14 xanh sau khi sửa . . . 36
28 CI xanh 4/4 job (api, api-e2e, web, e2e-browser) sau khi merge PR #22 . . . 37
29 CD từ tag `v0.1.1`: các bước quality, build-and-push, deploy-and-smoke-test đều xanh . . . 37
30 CD từ tag `v0.1.2` (commit `7b1ea96`) xanh . . . 38
31 Render Events: deploy image `sha-7b1ea96` qua Deploy Hook . . . 38
32 CD #11: bỏ qua build, deploy lại image cũ . . . 38
33 Log deploy `sha-e445b63` . . . 39
34 Render Events của lần quay lui . . . 39
35 Giao diện bản cũ sau quay lui . . . 40
36 CD #12: triển khai lại `sha-7b1ea96` . . . 40
37 Giao diện bản mới . . . 40

List of Tables

3 Đối chiếu rút gọn giữa Task của đề và chương báo cáo 8
4 Công nghệ và công cụ 8
5 Vai trò và trách nhiệm 9
6 Sprint 1: phạm vi và người nghiệm thu 10
7 Sprint 2: phạm vi và người nghiệm thu 10
8 Stakeholder 11
9 Yêu cầu chức năng và tiêu chí chấp nhận 12
10 Yêu cầu phi chức năng và cách kiểm chứng 13
11 Đặc tả UC-01: Đặt và thanh toán đơn hàng 15
12 Luồng thay thế (A) và ngoại lệ (E) của UC-01 16
13 Chuyển trạng thái hợp lệ 19
14 Thành phần (container) 26
15 Danh sách ADR (đều ở trạng thái đã chốt; ADR-007 chọn ngày 23/10/2026) 26
16 Các endpoint chính 29
17 Số lượng test theo mức 41
18 Test case chính và yêu cầu liên quan 41
19 Kiểm thử chức năng thủ công 42
20 Danh mục bàn giao 43
21 Kết quả Sprint Review Sprint 2 43
22 Hành động sau Sprint 44
23 Ma trận truy vết yêu cầu 44

Danh mục từ viết tắt

Viết tắt	Nghĩa đầy đủ
ADR	Architecture Decision Record (bản ghi quyết định kiến trúc)
API	Application Programming Interface
CD	Continuous Delivery / Deployment (phân phối, triển khai liên tục)
CI	Continuous Integration (tích hợp liên tục)
CSDL	Cơ sở dữ liệu
DoD	Definition of Done (định nghĩa hoàn thành)
E2E	End-to-End (kiểm thử đầu cuối)
ERD	Entity Relationship Diagram (sơ đồ thực thể – quan hệ)
FR / NFR	Functional / Non-functional Requirement (yêu cầu chức năng / phi chức năng)
GHCR	GitHub Container Registry
JWT	JSON Web Token
MVP	Minimum Viable Product (sản phẩm khả dụng tối thiểu)
PO	Product Owner
PR	Pull Request
SRS	Software Requirements Specification (đặc tả yêu cầu phần mềm)
UC	Use Case
UI	User Interface (giao diện người dùng)
UML	Unified Modeling Language

Bảng thuật ngữ

Thuật ngữ (Việt)	Tiếng Anh	Định nghĩa ngắn
Idempotency-Key	Idempotency key	Khóa do client gửi kèm yêu cầu để gửi lại cùng yêu cầu không tạo kết quả trùng.
Quay lui	Rollback	Triển khai lại một phiên bản ổn định trước đó khi bản mới có sự cố.
Kiểm thử khói	Smoke test	Kiểm tra nhanh sau triển khai rằng dịch vụ còn sống (ở đây là gọi `/health`).
Máy trạng thái	State machine	Tập trạng thái của đơn hàng và các chuyển trạng thái hợp lệ.
Nghiệm thu chéo	Cross acceptance	Người khác với người viết task kiểm tra task đạt DoD.

Tóm tắt

Báo cáo trình bày Assignment 2 của dự án BrewLite, đi từ yêu cầu đến triển khai: khai thác và đặc tả yêu cầu (17 yêu cầu chức năng, 11 yêu cầu phi chức năng), mô hình hóa bằng UML, thiết kế kiến trúc kèm bảy quyết định kiến trúc, và hai Sprint hiện thực (20–26/10/2026). Sản phẩm là ứng dụng đặt đồ uống dùng Next.js, NestJS, PostgreSQL, có pipeline CI bốn job, pipeline CD từ tag phiên bản lên GHCR và Render, kèm smoke test. Nhóm đã chứng minh CI chặn merge khi test lỗi và quay lui thành công từ phiên bản `v0.1.2` về bản trước rồi triển khai lại. Ma trận truy vết nối yêu cầu với thiết kế, mã nguồn và test. Còn mở: dấu vết nghiệm thu chéo trên GitHub, và các giới hạn của gói hạ tầng miễn phí (web tạm ngủ, CSDL hết hạn 22/11/2026).

Từ khóa: yêu cầu phần mềm; UML; kiến trúc; CI/CD; triển khai; truy vết

1 Giới thiệu và quy trình phát triển

Bảng 3 cho biết yêu cầu của Assignment 2 nằm ở chương nào; bản đầy đủ ở mục A.

Table 3: Đối chiếu rút gọn giữa Task của đề và chương báo cáo

Task của đề	Chương
Task 1: khai thác và đặc tả yêu cầu	mục 3
Task 2: mô hình hóa (activity, sequence, class)	mục 4
Task 3: kiến trúc và triển khai (deployment)	mục 5
Task 4: Sprint 1, repository, MVP, CI	mục 2, mục 6
Task 5: Sprint 2, CI/CD, bàn giao	mục 6, mục 7, mục 8
Truy vết yêu cầu	mục 9

1.1 Bối cảnh, mục tiêu, phạm vi

Quán cà phê quy mô nhỏ cần để khách đặt món và thanh toán không dùng tiền mặt ngay trên điện thoại, tránh xếp hàng và nhầm đơn. BrewLite là ứng dụng web mobile-first cho phép xem menu, chọn size và topping, quản lý giỏ hàng, đăng nhập, đặt đơn, thanh toán mô phỏng và xem lịch sử đơn.

Assignment 2 tiếp nối Assignment 1 và đi xuyên suốt từ khai thác yêu cầu, mô hình hóa, thiết kế kiến trúc, hiện thực hai Sprint, CI/CD đến bàn giao bản production. Mã nguồn dùng chung với Assignment 1; báo cáo này tập trung vào đặc tả, thiết kế, quy trình và vận hành. Phạm vi chi tiết (trong, ngoài, không thực hiện) ở mục 3.1.

1.2 Mô hình quy trình áp dụng

Nhóm áp dụng Agile Scrum [1]: Product Backlog, Sprint Backlog, Increment; bốn sự kiện Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective. Scrum phù hợp vì thời gian ngắn (khoảng một tuần), yêu cầu có thể điều chỉnh sau mỗi Sprint và cần sản phẩm chạy được sớm để triển khai thử. Assignment 2 chia thành hai Sprint: Sprint 1 (20–23/10) tạo nền tảng và MVP menu, Sprint 2 (24–26/10) hoàn thiện luồng end-to-end, CI/CD và bàn giao (xem mục 2.3).

1.3 Công nghệ và công cụ sử dụng

Table 4: Công nghệ và công cụ

Lớp	Công nghệ
Frontend	Next.js (App Router), mobile-first, manifest PWA cơ bản
Backend	NestJS, class-validator, JWT, bcrypt
Cơ sở dữ liệu	PostgreSQL 16, Prisma 7 (adapter `pg`), migration và seed
Kiểm thử	Vitest, Supertest (PostgreSQL thật), Playwright
Vận hành	Docker Compose, GitHub Actions, GHCR, Render
Sơ đồ	PlantUML (nguồn trong `docs/diagrams`)

Kho mã nguồn: https://github.com/tthan-prog/BrewLite. Web production: https://brewlite-web.onrender.com. API: https://brewlite-api.onrender.com/health. Phiên bản bàn giao: tag v0.1.2 (commit 7b1ea96, image sha-7b1ea96).

1.4 Cấu trúc báo cáo

mục 2 trình bày quản lý dự án; mục 3 đặc tả yêu cầu; mục 4 mô hình UML; mục 5 kiến trúc; mục 6 hiện thực và CI/CD; mục 7 kiểm thử; mục 8 bàn giao và nhìn lại; mục 9 ma trận truy vết; mục 10 kết luận. Phụ lục gồm bảng đối chiếu đề bài, phân công và tư liệu bổ sung.

2 Quản lý dự án (Scrum)

2.1 Vai trò / trách nhiệm trong nhóm

Vai trò theo docs/chung/team-agreement.md. Đây là "người chịu trách nhiệm chính", không phải chỉ một người làm. Scrum Guide 2020 gọi các vai là *accountabilities*; báo cáo dùng từ "vai trò" theo đề bài.

Table 5: Vai trò và trách nhiệm

Thành viên	Vai trò	Trách nhiệm cụ thể
Nguyễn Lê Nhựt Thắng (A)	Scrum Master, phát triển backend và nền tảng	Repo, CI/CD, triển khai, điều phối Scrum; API sản phẩm, xác thực, thanh toán; sequence diagram
Hồ Thiên Ngân (B)	Backend	Dữ liệu, seed, tạo đơn, trạng thái đơn
Hà Thiên Long (C)	Kiểm thử, Product Owner tạm thời	Tài liệu yêu cầu, truy vết, backlog; unit, tích hợp, E2E
Giản Thiên Lực (D)	Frontend	Menu, giỏ, thanh toán, lịch sử; activity, use-case; slide, video

2.2 Sự kiện và artefact

Sprint Planning đầu Sprint (20/10 và 24/10), Daily Scrum hằng ngày tối đa 15 phút qua nhóm chat, Sprint Review và Retrospective cuối Sprint (24/10 và 26/10). Artefact: Product Backlog, Sprint Backlog (kèm Sprint Goal) và Increment là bản chạy được trên production. Biên bản nằm ở docs/as2/scrum/minutes/a2-*. Quy ước chung: không đẩy trực tiếp lên main; mỗi chức năng một nhánh; PR phải qua check api và web, cần một approval; đóng băng mã tối 25/10.

2.3 Kế hoạch và diễn biến từng Sprint

2.3.1 Sprint 1 (20–23/10/2026)

Sprint Goal: tạo nền tảng kỹ thuật, tài liệu có kiểm soát phiên bản và MVP đầu tiên cho luồng xem menu. Planning họp ngày 20/10.

Table 6: Sprint 1: phạm vi và người nghiệm thu

Task	Nội dung	Làm chính	Nghiệm thu
4.1	Repo, cấu trúc, README, `.env.example`, branch protection, quy ước commit, DoD	A	B
4.2	Tạo phẩm Task 1–3 trong `docs`, tag `assign1-final`, thay đổi qua PR	A; C (viết yêu cầu)	D
4.3	Dữ liệu mẫu, `GET /products`, `GET /products/:id`	B (dữ liệu), A (API)	C
4.3	Màn hình menu mobile-first, loading, empty, error	D	A
4.3	Kiểm thử tối thiểu API và menu	C	A
4.4	Pipeline CI, chặn merge khi đỏ	A	C

Kết quả Review (24/10): cả bốn hạng mục demo được và đạt DoD, Sprint Goal đạt. Điểm còn lại cuối Sprint là 0 trên 4 việc; repo chưa có file burndown riêng cho Assignment 2 nên số này tính theo số việc, không phải story point.

2.3.2 Sprint 2 (24–26/10/2026)

Sprint Goal: hoàn thành luồng end-to-end, tự động hóa đóng gói và triển khai, có bản production truy cập được. Planning họp ngày 24/10.

Table 7: Sprint 2: phạm vi và người nghiệm thu

Task	Nội dung	Làm chính	Nghiệm thu
5.1	Đăng ký, đăng nhập, thanh toán mô phỏng	A	B
5.1	Tạo đơn, trạng thái đơn	B	C
5.1	Giao diện menu, giỏ, thanh toán, lịch sử	D	A
5.2	Unit, integration, E2E, Docker Compose, `/health`	C	B
5.3	Pipeline CD, rollback	A	B
5.4	Triển khai production, release `v0.1.x`, README	A	D
5.4	Video demo end-to-end	D	C
5.4	Sprint Review và Retrospective	A	Cả nhóm

Kết quả Review (24/10, cập nhật tới 26/10): Task 5.1, 5.2 và 5.3 demo được và đạt DoD; Task 5.4 "gần đạt"; video demo chưa xong tại thời điểm Review và được hoàn tất sau đó (xem mục 8.1). Điểm còn lại lúc Review là 2 việc (video), tính theo số việc.

2.4 Definition of Done

Bốn mục DoD áp dụng cho mọi task (`docs/as1/scrum/dod-nghiem-thu.md`): (D1) chạy được, không lỗi build; (D2) commit rõ ràng và PR có ít nhất một Approve của người khác; (D3) API validate và giao diện có loading, lỗi; (D4) README có hướng dẫn chạy. Người nghiệm thu khác người viết task.

Mức độ áp dụng trung thực: D1, D3, D4 có bằng chứng (CI xanh, test, README). Với D2, nhóm ghi nhận nghiệm thu chéo trong tài liệu, nhưng các PR chưa có Approve hoặc bình luận nghiệm thu của người đánh giá trên GitHub (cột Reviews trống). Điểm này đã được nêu ở Retrospective của cả hai Sprint.

3 Đặc tả yêu cầu

Đặc tả đầy đủ nằm ở `docs/requirements/srs.md`, `context-and-scope.md` và `use-case-checkout.md`. Hệ mã FR/NFR được giữ xuyên suốt để lập ma trận truy vết ở mục 9.

3.1 Bối cảnh, phạm vi (trong / ngoài)

Vấn đề nghiệp vụ. Quán cà phê quy mô nhỏ cần để khách đặt món và thanh toán không dùng tiền mặt trên điện thoại, tránh xếp hàng và nhầm đơn.

Trong phạm vi MVP: xem menu và chi tiết (size S/M/L, topping); giỏ hàng; đăng ký, đăng nhập, JWT, phân quyền CUSTOMER và STAFF; tạo đơn; thanh toán Ví hoặc Thẻ mô phỏng; xác nhận, theo dõi trạng thái, lịch sử đơn; máy trạng thái, thanh toán idempotent, kiểm soát tồn kho nguyên liệu đồng thời, khuyến mãi, điểm thưởng; triển khai production qua CI/CD.

Mở rộng (nếu còn thời gian): PWA đầy đủ (offline, thông báo); giao diện quản trị; giao diện riêng cho nhân viên quầy. **Không thực hiện:** cổng thanh toán thật, ứng dụng native, giao hàng và nhiều chi nhánh, quản lý kho nguyên liệu đầy đủ (nhập theo lô, nhà cung cấp, kiểm kê); hệ thống chỉ có kho nguyên liệu và công thức ở mức cơ bản.

3.2 Stakeholder

Table 8: Stakeholder

Stakeholder	Vai trò	Mục tiêu và kỳ vọng	Ảnh hưởng đến yêu cầu
Khách hàng	Đặt và thanh toán đơn	Đặt nhanh, giá rõ, theo dõi được đơn	Menu, giỏ, thanh toán, lịch sử
Nhân viên quầy	Pha chế, cập nhật trạng thái	Nhận đúng đơn đã thanh toán	`PATCH /orders/:id/status`
Quản lý cửa hàng	Quản lý sản phẩm, khuyến mãi	Kiểm soát tồn kho, khuyến mãi	Tồn kho, khuyến mãi; quản trị ngoài MVP
Giảng viên	Khách hàng nghiệm thu	Chạy được, có tài liệu và bằng chứng	Toàn bộ sản phẩm bàn giao
Nhóm phát triển	Xây dựng sản phẩm	Đúng hạn, dễ bảo trì	Ràng buộc công nghệ, CI/CD
Đơn vị thanh toán (mô phỏng)	Xử lý Ví/Thẻ	Nhận yêu cầu hợp lệ, không trừ tiền lặp	Idempotency, trạng thái thanh toán

3.3 Giả định, ràng buộc, phụ thuộc

Giả định. (G1) Thanh toán mô phỏng, có thể thành công hoặc thất bại theo kịch bản. (G2) Một cửa hàng, một loại tiền (VND). (G3) Phụ phí size: S +0, M +5.000, L +10.000 đồng. (G4) Điểm thưởng: 1 điểm cho mỗi 10.000 đồng. (G5) Tồn kho tính theo nguyên liệu (`Ingredient.stock`) cho món có công thức và

trừ khi thanh toán thành công; món chưa có công thức dùng `Product.stock` và trừ khi tạo đơn; hủy đơn chỉ hoàn phần đã trừ. (G6) Đơn mới ở trạng thái PENDING.

Ràng buộc. (R1) Bắt buộc Next.js, NestJS, PostgreSQL. (R2) Frontend mobile-first/PWA. (R3) Hạn nộp theo thông báo của giảng viên. (R4) Nhóm bốn người, làm theo Scrum. (R5) Production qua HTTPS, CSDL không public, secret không nằm trong repo.

Rủi ro ban đầu. Không kịp triển khai production (giảm thiểu: triển khai thử sớm, 23/10); hai bài tập lớn chồng lịch (dùng chung mã nguồn); đặt hoặc thanh toán đồng thời gây âm tồn kho (cập nhật có điều kiện trong transaction, có test); sơ đồ mâu thuẫn SRS (rà bằng ma trận truy vết); thành viên chậm tiến độ (quy trình trong thỏa thuận nhóm).

3.4 Yêu cầu chức năng

Mức ưu tiên theo MoSCoW. FR-16 (Could) và FR-17 (Won't) nằm ngoài MVP.

Table 9: Yêu cầu chức năng và tiêu chí chấp nhận

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận
FR-01	Hệ thống hiển thị danh sách sản phẩm có tên, giá, ảnh (nếu có)	Must	`GET /products` trả danh sách; màn hình menu hiển thị đủ sản phẩm có trong seed
FR-02	Hệ thống hiển thị các trạng thái loading, rỗng và lỗi của menu	Must	Mô phỏng API chậm, trả rỗng, trả lỗi thì UI hiện đúng từng trạng thái
FR-03	Xem chi tiết sản phẩm; chọn size (S/M/L) và topping	Must	`GET /products/:id` trả đủ tùy chọn; giá cập nhật theo lựa chọn (S +0, M +5.000, L +10.000)
FR-04	Thêm, sửa số lượng, xóa món; hiển thị tổng tiền	Must	Tổng tiền = Σ (đơn giá + phụ phí size + topping) $\times$ số lượng
FR-05	Đăng ký và đăng nhập; phát JWT; mật khẩu băm bằng bcrypt	Must	Đăng nhập đúng nhận JWT; sai mật khẩu trả 401; DB không lưu mật khẩu thô
FR-06	Chặn đặt đơn khi chưa đăng nhập	Must	`POST /orders` không có JWT trả 401
FR-07	Tạo đơn từ giỏ hàng, trạng thái PENDING; kiểm tra đủ nguyên liệu cho cả đơn (món có công thức) hoặc trừ `Product.stock` trong cùng transaction (món chưa có công thức)	Must	Đơn tạo đúng; thiếu nguyên liệu hoặc hết hàng thì từ chối (400), không tạo đơn; với món có công thức, tạo đơn chưa trừ nguyên liệu
FR-08	Thanh toán Ví hoặc Thẻ mô phỏng; thành công $\rightarrow$ PAID, thất bại $\rightarrow$ PAYMENT_FAILED (API trả 200 kèm trạng thái đơn); thử lại được	Must	`POST /payments` đổi trạng thái đúng; thất bại giữ nguyên giỏ hàng
FR-09	Gửi lại yêu cầu thanh toán với cùng Idempotency-Key không tạo thêm thanh toán/đơn	Must	Hai request cùng khóa cho một kết quả và một bản ghi
FR-10	Máy trạng thái đơn; chặn chuyển sai trạng thái	Must	PENDING $\rightarrow$ PAID/PAYMENT_FAILED/CANCELLED; PAID $\rightarrow$ PREPARING/CANCELLED; PREPARING $\rightarrow$ READY $\rightarrow$ COMPLETED; chuyển sai trả lỗi
FR-11	Nhân viên cập nhật trạng thái đơn	Should	`PATCH /orders/:id/status` chỉ vai trò STAFF thực hiện được
FR-12	Khách xem lịch sử và chi tiết đơn của mình	Must	`GET /orders/me` chỉ trả đơn của người dùng hiện tại; `GET /orders/:id` chỉ chủ đơn hoặc STAFF xem được (người khác nhận 404)

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận
FR-13	Nguyên liệu chỉ bị trừ khi thanh toán thành công (cập nhật có điều kiện trong cùng transaction); đồng thời không làm tồn kho âm; hủy đơn chỉ hoàn phần đã trừ	Must	Test N request đồng thời, số đơn thành công $\leq$ tồn kho; kho hết đúng lúc thanh toán thì không trừ tiền, không trừ kho, đơn chuyển PAYMENT_FAILED
FR-14	Áp mã khuyến mãi hợp lệ để giảm giá	Should	Mã hợp lệ giảm đúng; mã sai/hết hạn bị từ chối
FR-15	Cộng điểm khi đơn thanh toán thành công (PAID): 1 điểm / 10.000đ; hủy đơn đã PAID thì trừ lại; mỗi lần đổi điểm ghi một giao dịch điểm (`LoyaltyTransaction`)	Should	`User.points` tăng đúng số điểm khi đơn PAID và giảm lại khi hủy đơn đã PAID; có bản ghi `LoyaltyTransaction` loại EARN (và REVERSAL khi hủy) tương ứng
FR-16	Giao diện quản lý sản phẩm và khuyến mãi	Could	Ngoài MVP
FR-17	Kết nối cổng thanh toán thật	Won't	Không thực hiện

3.5 Yêu cầu phi chức năng

Table 10: Yêu cầu phi chức năng và cách kiểm chứng

Mã	Yêu cầu	Ưu tiên	Tiêu chí chấp nhận
NFR-01	`GET /products` phản hồi nhanh với dữ liệu seed	Should	Lần chậm thứ 19 trong 20 yêu cầu `GET /products` dưới 500 ms với dữ liệu mẫu (12 sản phẩm), đo cục bộ bằng Docker Compose. Kết quả ngày 24/10/2026: 19,7 ms. Ghi chú production (Render Singapore, gói Free, đo từ Việt Nam, 30 yêu cầu): byte đầu tiên về sau 0,43 đến 0,47 giây ở 24 yêu cầu và 0,74 đến 0,78 giây ở 6 yêu cầu, trong đó khoảng 0,25 giây là độ trễ mạng một vòng
NFR-02	Mật khẩu băm; JWT có hạn; kiểm tra phân quyền theo vai trò	Must	Test xác thực và phân quyền; không có mật khẩu thô trong DB hoặc log
NFR-03	Secret không nằm trong repository hoặc log CI	Must	Quét repo/log không thấy secret; `.env` nằm trong `.gitignore`
NFR-04	Validate dữ liệu đầu vào ở cả frontend và backend	Must	Dữ liệu sai trả lỗi 4xx rõ ràng
NFR-05	Giao diện mobile-first, dùng được trên màn hình điện thoại	Must	Kiểm tra trên viewport 360–414 px, không cuộn ngang
NFR-06	Thông báo lỗi dễ hiểu, không lộ chi tiết kỹ thuật	Should	Lỗi hiển thị bằng tiếng Việt, không in stack trace
NFR-07	Chạy toàn bộ hệ thống bằng `docker compose up`	Must	Người ngoài nhóm chạy được theo README
NFR-08	CI chạy lint, test, build cho mọi PR; pipeline CD triển khai và smoke test	Must	Có log pipeline thành công và một lần phát hiện lỗi
NFR-09	`GET /health` và log để vận hành	Must	Health trả `status: ok`; smoke test gọi health sau deploy
NFR-10	Migration và seed tái tạo được database	Must	Chạy migration + seed trên DB trống cho kết quả giống nhau
NFR-11	Mã nguồn có lint, định dạng và test tự động	Should	CI chạy lint và test

Kết quả kiểm chứng thực tế của từng NFR nằm ở mục 9 và mục 7; NFR-01 đo thủ công, NFR-03 mới dò sơ bộ.

3.6 Use case và đặc tả use case

Sơ đồ use-case tổng quát ở Hình 1; phạm vi UC-01 ở Hình 2. UC-01 là luồng sinh doanh thu, sai ở đây gây mất tiền hoặc sai tồn kho, nên là use case trọng tâm. Hai sơ đồ theo ký hiệu chuẩn: ranh giới hệ thống là `BrewLite`, actor nối với use case bằng đường liền, tên use case chỉ nêu việc cần làm (không nêu cách làm), `<<include>>` trỏ từ use case gốc sang use case được bao gồm (Đăng nhập), `<<extend>>` trỏ từ use case mở rộng sang use case gốc và ghi điều kiện kích hoạt (khách nhập mã, thanh toán thất bại, thanh toán thành công). Hủy đơn là use case độc lập của nhân viên.

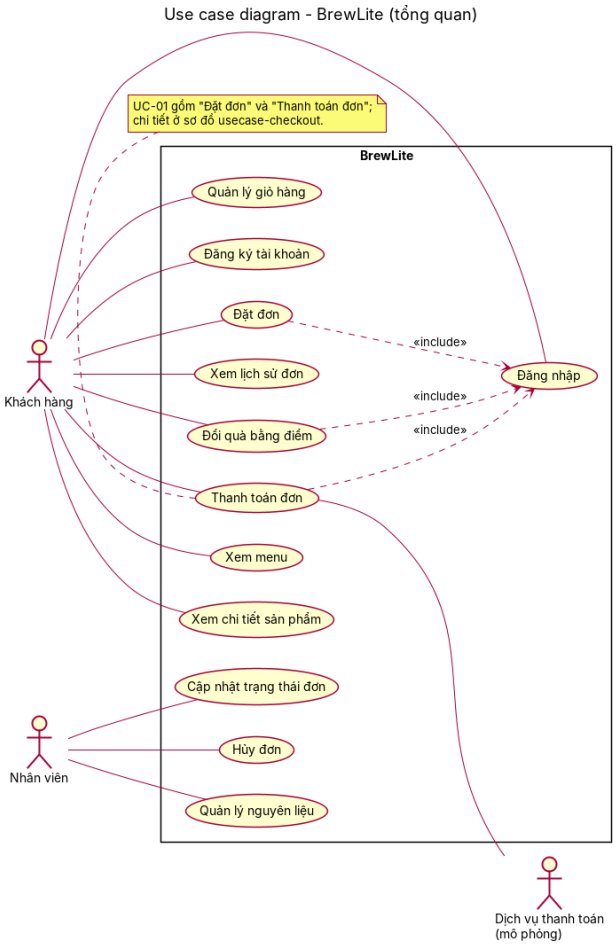


Figure 1: Use case tổng quan của BrewLite

Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/usecase-overview.puml`)
Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/usecase-checkout.puml`)

Luồng chính. (1) Khách mở giỏ và bấm "Đặt đơn". (2) Giao diện gửi `POST /orders` kèm JWT. (3) API kiểm tra JWT và validate. (4) API mở transaction: kiểm tra đủ nguyên liệu cho cả đơn và ghi nhu cầu vào `OrderIngredientUse` (món chưa có công thức thì trừ `Product.stock` có điều kiện), tạo

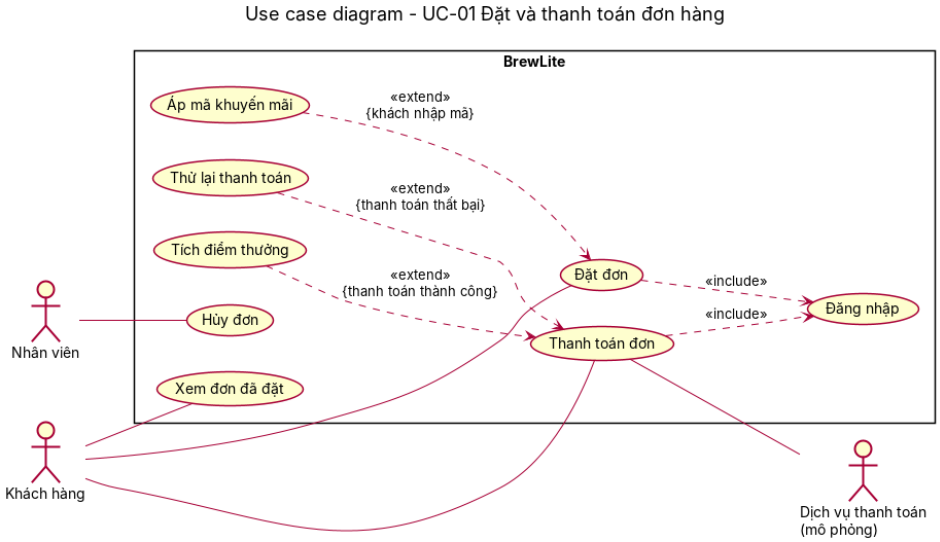


Figure 2: Use case của phạm vi UC-01

Table 11: Đặc tả UC-01: Đặt và thanh toán đơn hàng

Mã, tên	UC-01, Đặt và thanh toán đơn hàng
Actor chính / phụ	Customer / Payment Service (mô phỏng)
Trigger	Khách bấm "Đặt đơn" trong giỏ hàng
Yêu cầu liên quan	FR-04 đến FR-10, FR-13 đến FR-15, NFR-02, NFR-04
Tiền điều kiện	Khách có JWT hợp lệ; giỏ có ít nhất một món
Hậu điều kiện	Thành công: đơn PAID, nguyên liệu đã trừ khi thanh toán, có bản ghi thanh toán. Thanh toán thất bại: PAYMENT_FAILED, giữ giỏ, cho thử lại. Hết hàng hoặc lỗi: không tạo đơn, tồn kho không đổi; kho hết đúng lúc thanh toán: không trừ tiền, không trừ kho, đơn PAYMENT_FAILED

Order (PENDING) và OrderItem, áp khuyến mãi nếu có, commit; chưa trừ nguyên liệu của món có công thức. (5) API trả mã đơn; giao diện hiển thị thanh toán. (6) Khách chọn Ví hoặc Thẻ. (7) Giao diện gửi `POST /payments` kèm `Idempotency-Key`. (8) Payment Service trả thành công. (9) Trong cùng transaction, API trừ nguyên liệu bằng cập nhật có điều kiện, ghi Payment và chuyển đơn sang PAID. (10) Giao diện hiển thị xác nhận.

Table 12: Luồng thay thế (A) và ngoại lệ (E) của UC-01

Mã	Điều kiện	Xử lý
A1	Mã khuyến mãi hợp lệ	Giảm giá, cập nhật tổng tiền trước khi đặt đơn
A2	Gửi lại thanh toán cùng `Idempotency-Key`	Trả kết quả cũ, không tạo thanh toán thứ hai
A3	STAFF hủy đơn ở trạng thái cho phép	CANCELLED, chỉ hoàn phần nguyên liệu đã trừ, trừ lại điểm thưởng
E1	Chưa đăng nhập hoặc JWT hết hạn	401, chuyển sang đăng nhập, giữ giỏ
E2	Dữ liệu không hợp lệ	400 kèm thông báo rõ ràng
E3	Món hết hàng hoặc không đủ số lượng	Rollback, không tạo đơn, báo món nào hết
E4	Mã khuyến mãi sai hoặc hết hạn	Từ chối mã, cho đặt đơn không mã
E5	Payment Service trả thất bại	PAYMENT_FAILED, giữ giỏ, cho thử lại
E6	Chuyển trạng thái không hợp lệ	400, đơn giữ nguyên trạng thái

4 Mô hình hóa (UML)

Nguồn PlantUML nằm ở `docs/diagrams`; ảnh dưới đây kết xuất từ đúng các tệp nguồn đó. Ba sơ đồ bắt buộc của đề (activity, sequence, class) cùng sơ đồ trạng thái đơn hàng được rà theo SRS và mã nguồn.

4.1 Sơ đồ hoạt động (Activity)

Hình 3 mô tả luồng đặt và thanh toán, tách thành hai phần cho dễ đọc (Hình 3 đặt đơn, Hình 4 thanh toán và xử lý đơn), theo bốn swimlane (Khách hàng, Hệ thống, Dịch vụ thanh toán, Nhân viên), mỗi hành động nằm đúng làn của người thực hiện. Sơ đồ có nút bắt đầu và kết thúc, nút quyết định kèm điều kiện trên từng nhánh, nút hợp nhất và hai vòng lặp: chỉnh giỏ hàng khi thiếu nguyên liệu, và thử lại khi thanh toán thất bại. Các nhánh bắt buộc có đủ: thiếu nguyên liệu, chưa đăng nhập, thanh toán thất bại, kho vừa hết lúc thanh toán. Liên quan: UC-01, FR-04 đến FR-08.

4.2 Sơ đồ tuần tự (Sequence)

UC-01 được mô tả bằng hai sơ đồ tuần tự, mỗi sơ đồ một kịch bản để dễ đọc: đặt đơn (Hình 5) và thanh toán (Hình 6). Lifeline xếp từ trái sang phải theo thứ tự tham gia, thanh chữ nhật mảnh trên lifeline là khoảng thời gian đối tượng đang xử lý (activation), thời gian chạy từ trên xuống dưới. Các tình huống rẽ nhánh dùng khung kết hợp: `break` cho nhánh lỗi thoát sớm, `loop` cho mỗi món trong đơn, `opt` cho bước tùy chọn và `alt` cho nhánh thay thế.

Sơ đồ đặt đơn gồm bốn lifeline: Khách hàng, Giao diện web (Next.js), API (`OrdersService`) và PostgreSQL. Sơ đồ thể hiện kiểm tra JWT và dữ liệu (401, 400), kiểm tra khóa idempotency (trả lại đơn cũ hoặc lỗi 409), kiểm tra nguyên liệu theo công thức trong một giao dịch và `ROLLBACK` khi thiếu, áp mã khuyến mãi, rồi lưu đơn ở trạng thái PENDING. Nguyên liệu chỉ được kiểm tra ở bước này; việc trừ kho diễn ra khi thanh toán thành công.

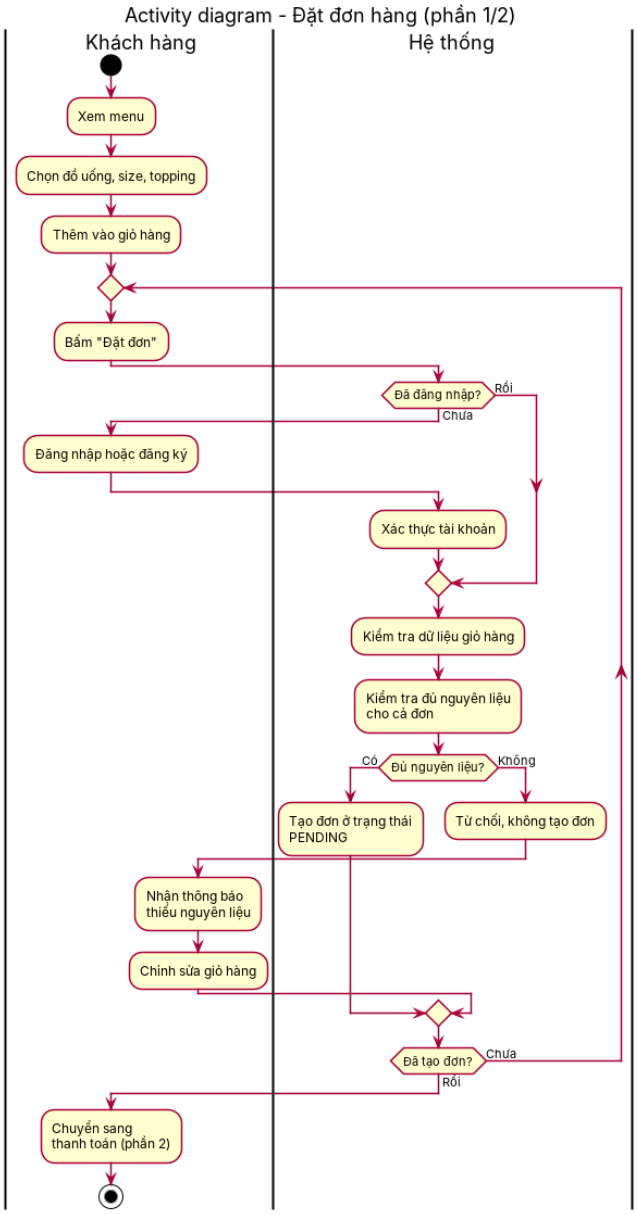


Figure 3: Sơ đồ hoạt động: đặt đơn (phần 1/2)

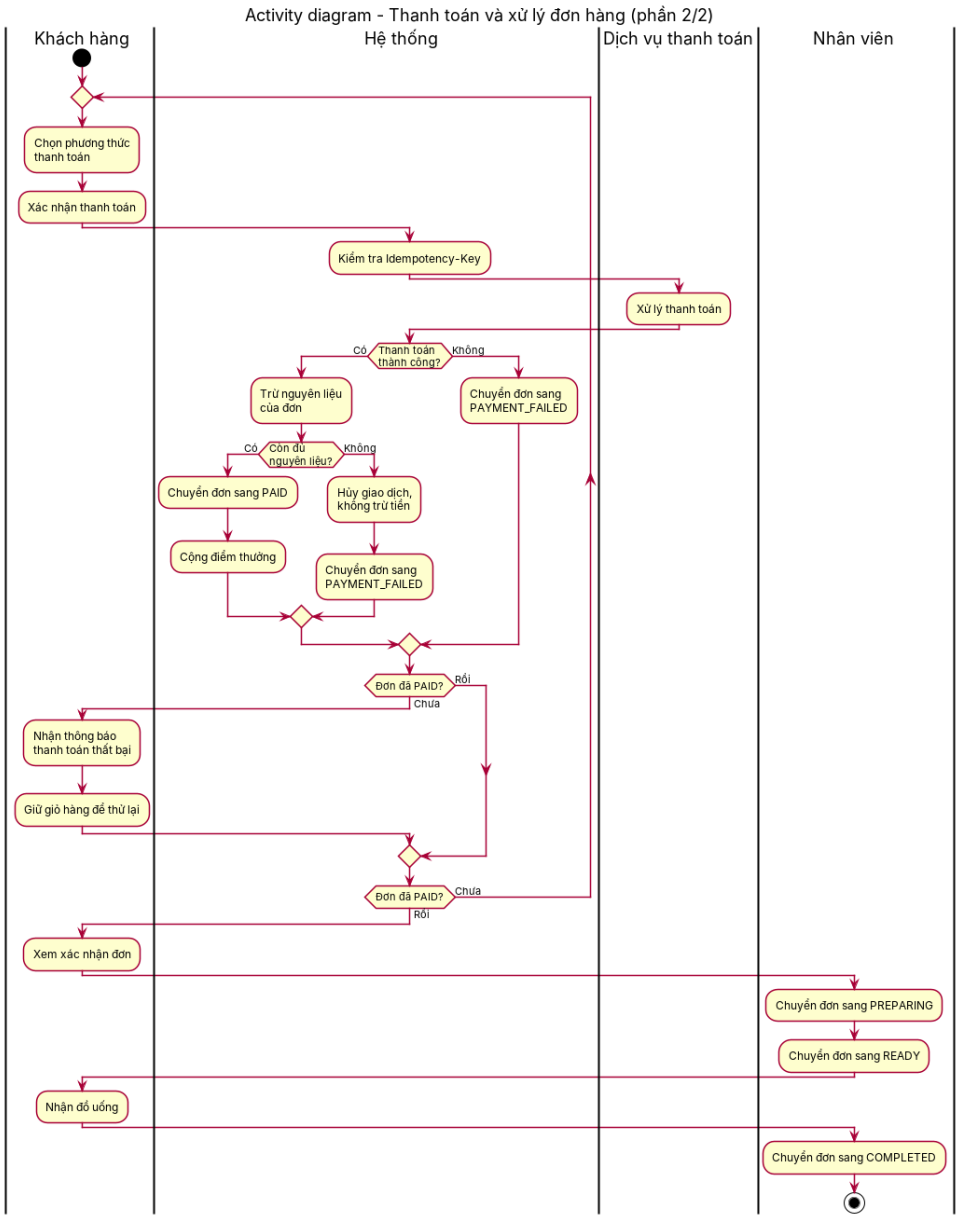


Figure 4: Sơ đồ hoạt động: thanh toán và xử lý đơn (phần 2/2)

Sơ đồ thanh toán gồm năm lifeline: thêm `OrdersService` để thể hiện việc đổi trạng thái đơn. Sơ đồ thể hiện kiểm tra khóa idempotency, kiểm tra đơn hợp lệ, thử lại từ PAYMENT_FAILED, thanh toán thành công kèm trừ nguyên liệu có điều kiện, nhánh kho vừa hết do đơn khác (`ROLLBACK`, không trừ tiền, không trừ kho, đơn chuyển PAYMENT_FAILED) và nhánh thanh toán thất bại. Liên quan: UC-01, FR-06 đến FR-09, FR-13 đến FR-15.

4.3 Sơ đồ lớp / mô hình miền (Class / Domain)

Hình 7, Hình 8 và Hình 9 khớp với `schema.prisma`, mỗi lớp có đủ ba ngăn (tên, thuộc tính, phương thức) với ký hiệu phạm vi truy cập (− riêng tư, + công khai). Hình 7 gồm các lớp đặt hàng và điểm thưởng: User, Product, Topping, Order, OrderItem, Promotion, LoyaltyTransaction, IdempotencyRecord. Hình 8 gồm thanh toán, phiên QR và quà đổi điểm: Payment, PaymentIntent, PaymentAccount, Reward, Redemption (cùng User và Order ở dạng rút gọn thuộc tính). Các kiểu liệt kê (OrderStatus, PaymentMethod, PaymentStatus, PaymentProvider, IntentStatus, PromotionType, LoyaltyType, Role) không vẽ thành hộp riêng mà ghi giá trị trong ghi chú cạnh lớp dùng chúng. Quan hệ gồm kết hợp có tên, chiều đọc và multiplicity, composition giữa Order và OrderItem (dòng đơn không tồn tại khi đơn bị xóa), và lớp kết hợp `OrderItemTopping` giữa OrderItem và Topping. Hình 9 gồm kho nguyên liệu và công thức, với ba lớp kết hợp RecipeItem, ToppingRecipeItem và OrderIngredientUse. Các phương thức là thao tác mức miền, ánh xạ tới hàm hoặc dịch vụ trong mã (ghi chú trên sơ đồ).

Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/class-domain.puml`)

Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/class-payment-reward.puml`)

Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/class-inventory.puml`)

4.4 Sơ đồ trạng thái (State machine)

Đơn hàng có vòng đời trạng thái (FR-10). Bảng 13 mô tả các chuyển trạng thái hợp lệ do `apps/api/src/orders/order-state.ts` thực thi; chuyển trạng thái khác bị từ chối với mã 400 và đơn giữ nguyên.

Table 13: Chuyển trạng thái hợp lệ

Từ	Sang
PENDING	PAID, PAYMENT_FAILED, CANCELLED
PAYMENT_FAILED	PAID (thử lại), CANCELLED
PAID	PREPARING, CANCELLED
PREPARING	READY
READY	COMPLETED

Khi hủy đơn, chỉ phần nguyên liệu đã trừ được hoàn lại (đơn chưa thanh toán không đụng tới kho); nếu đơn đã PAID thì điểm thưởng đã cộng bị trừ lại (REVERSAL).

5 Kiến trúc hệ thống

5.1 Kiến trúc tổng thể

Kiến trúc phân lớp kết hợp mức container của mô hình C4: trình duyệt/PWA → frontend Next.js → backend NestJS (modular monolith, module theo domain) → PostgreSQL; dịch vụ thanh toán mô phỏng là một module trong backend (Hình 10). Chi tiết ở `docs/architecture/architecture.md`.

Nguồn: nhóm vẽ bằng PlantUML (`docs/diagrams/architecture-container.puml`)

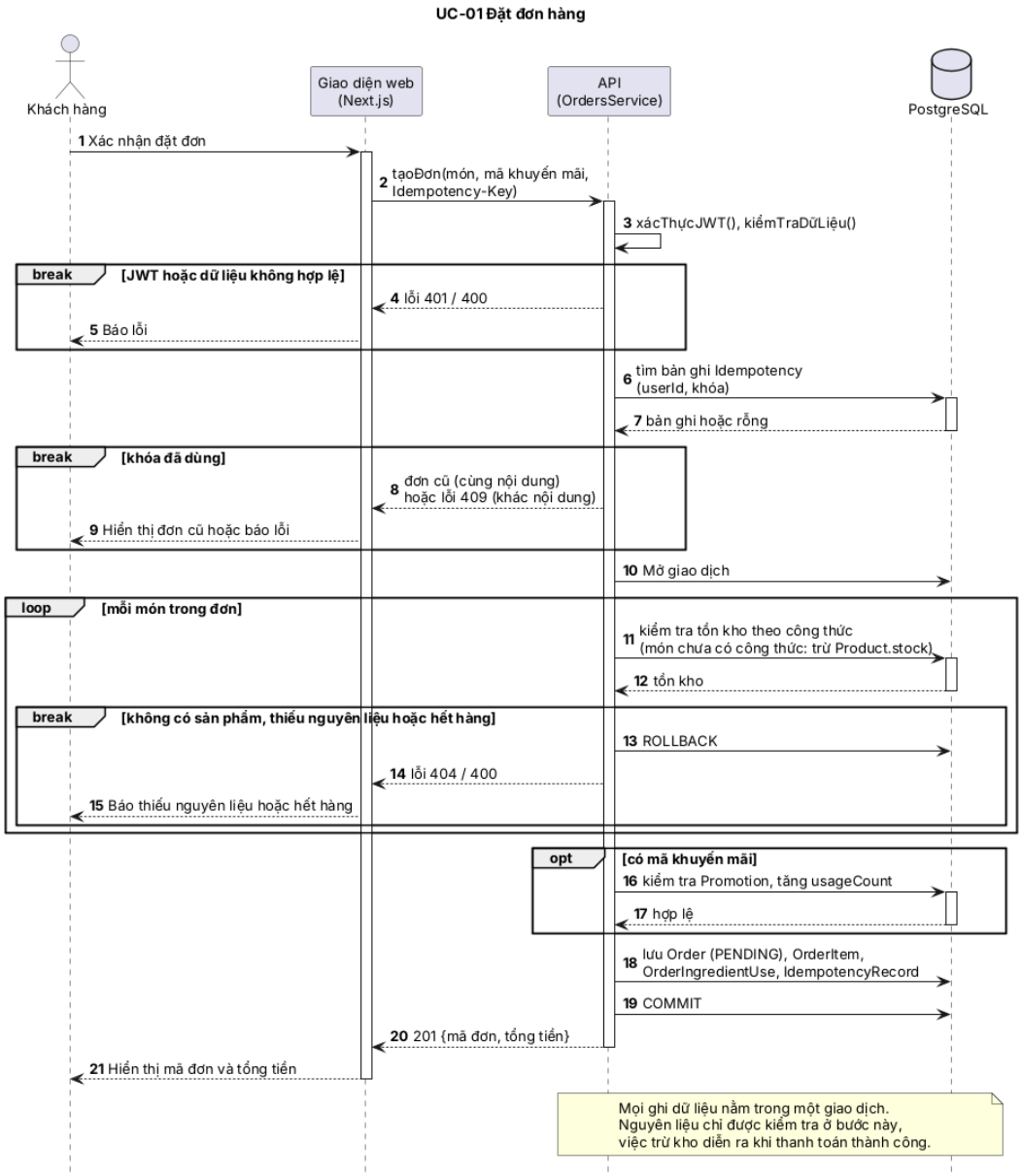


Figure 5: Sơ đồ tuần tự: đặt đơn (UC-01)

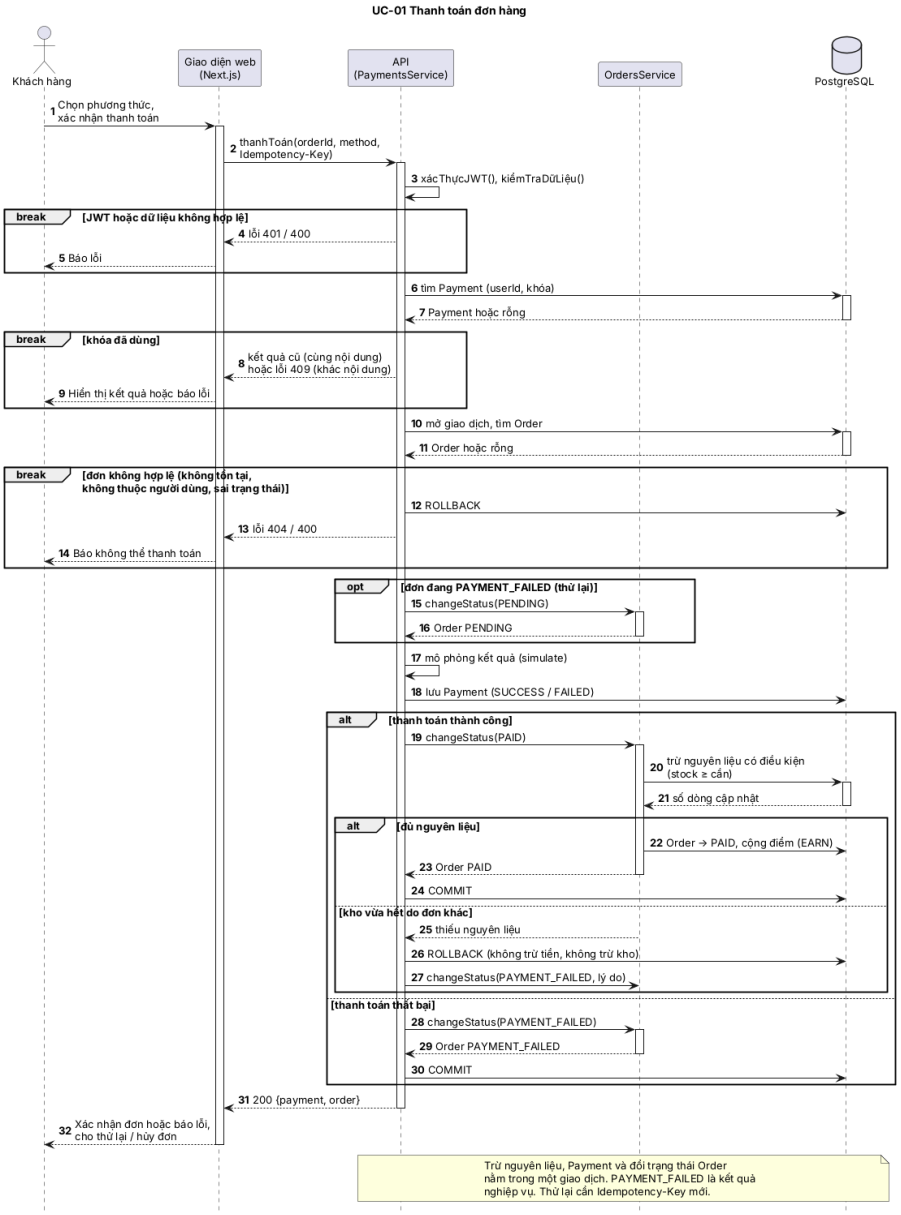


Figure 6: Sơ đồ tuần tự: thanh toán (UC-01)

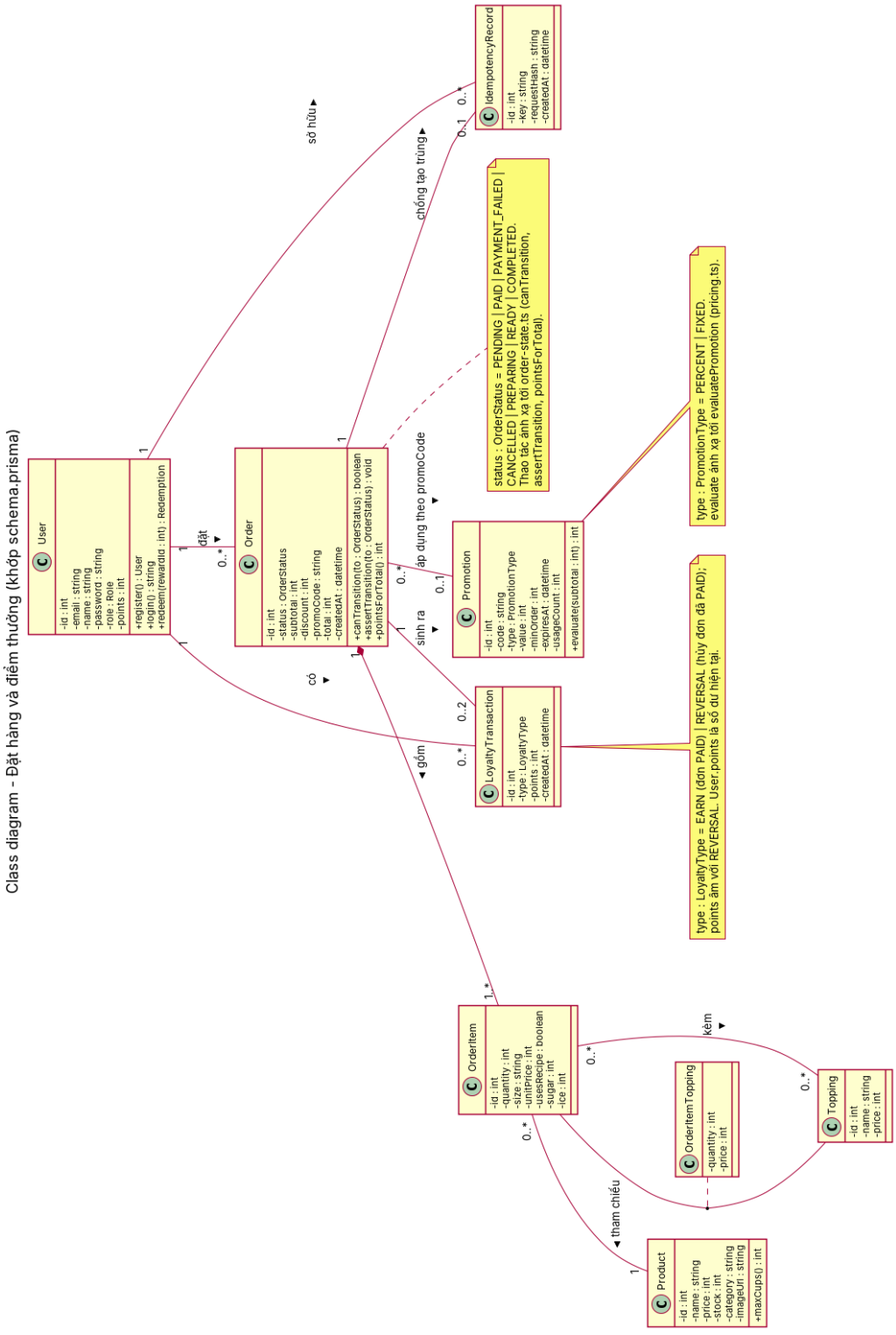


Figure 7: Sơ đồ lớp: đặt hàng và điểm thưởng

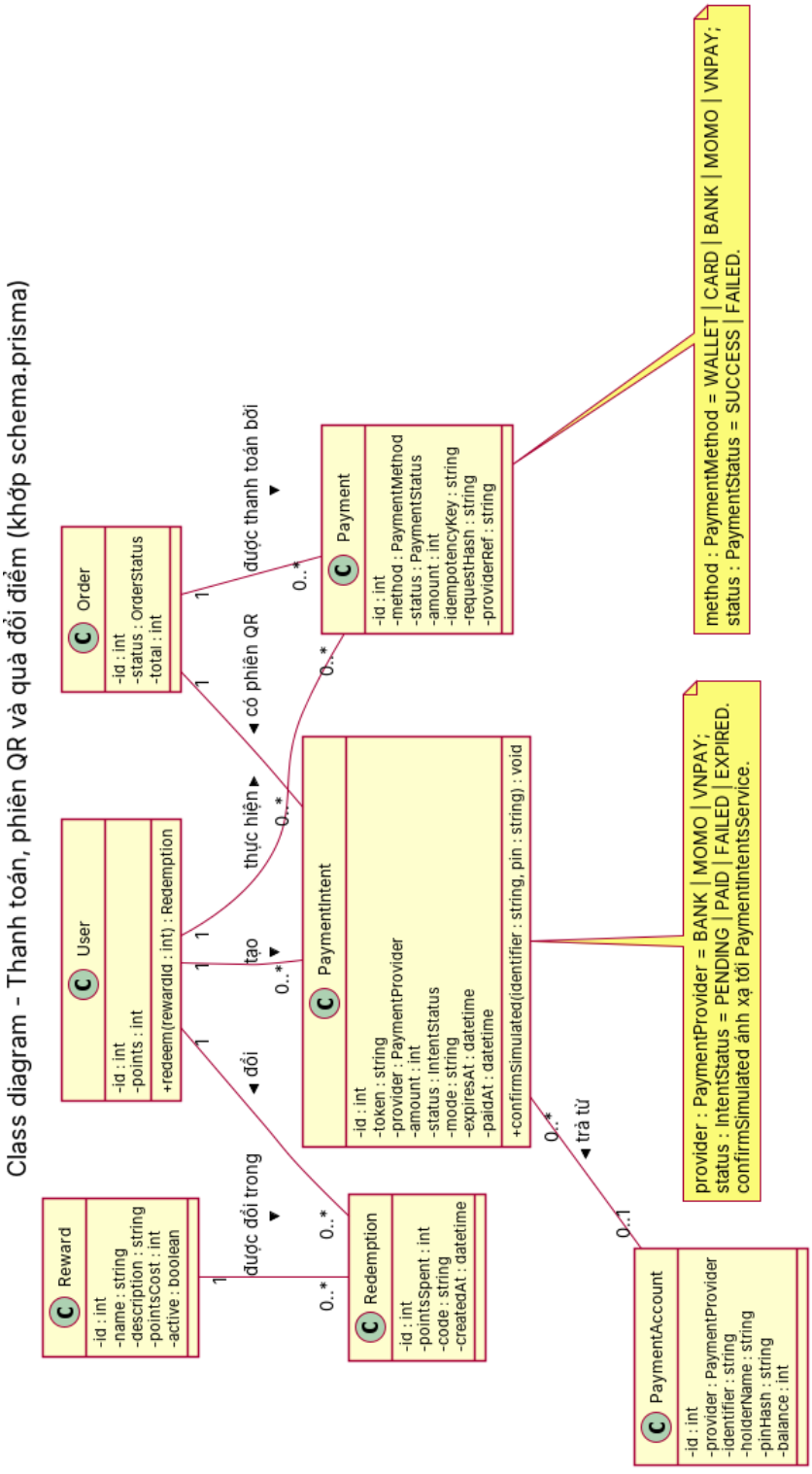


Figure 8: Sơ đồ lớp: thanh toán, phiên QR và quà đổi điểm

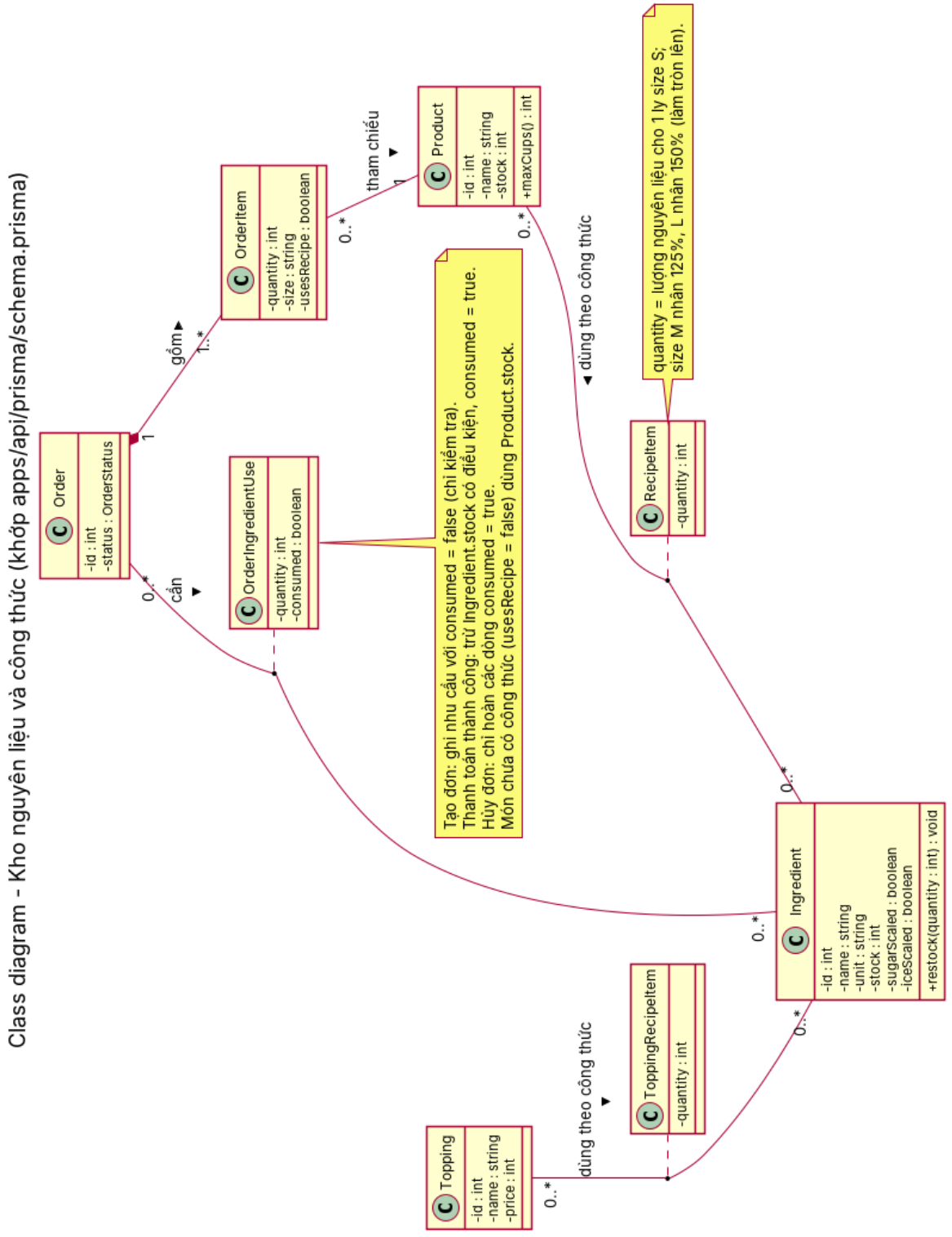


Figure 9: Sơ đồ lớp: kho nguyên liệu và công thức

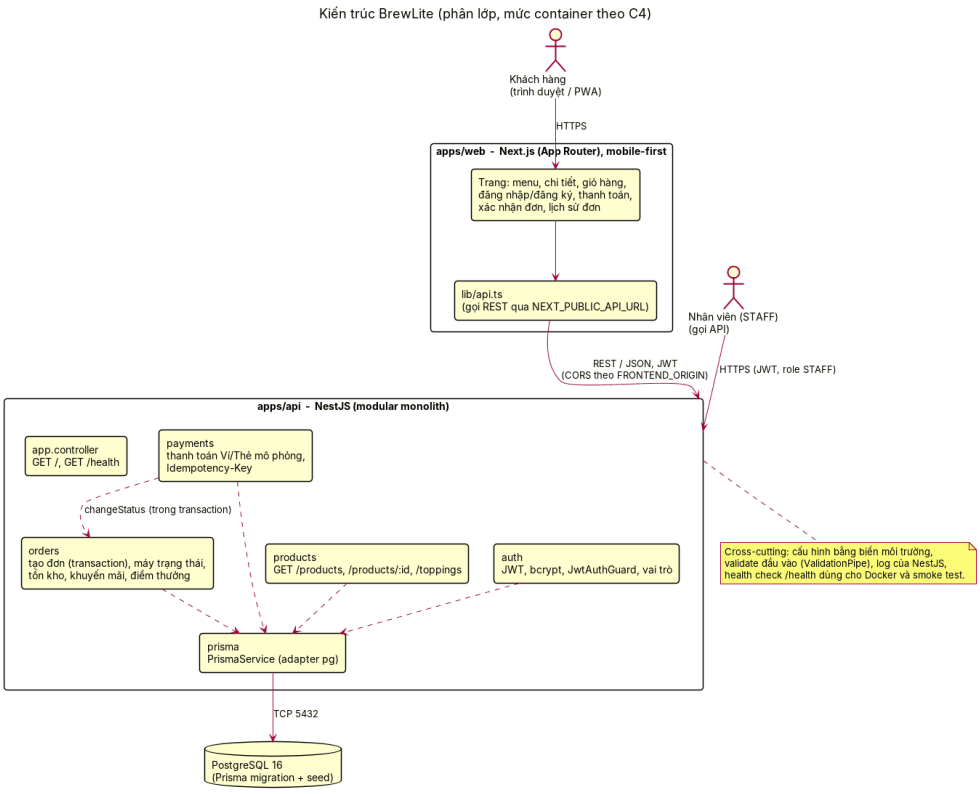


Figure 10: Kiến trúc mức container của BrewLite

Table 14: Thành phần (container)

Thành phần	Công nghệ	Trách nhiệm	Mã nguồn
Web	Next.js, mobile-first	Menu, chi tiết, giỏ, đăng nhập, xác nhận, lịch sử	`apps/web`
API	NestJS, JWT, validation	Nghiệp vụ, xác thực, phân quyền, truy cập CSDL	`apps/api`
CSDL	PostgreSQL 16, Prisma 7	Người dùng, sản phẩm, đơn, thanh toán, điểm, bản ghi idempotency	`apps/api/prisma`
Payment (mô phỏng)	Module NestJS	Ví/Thẻ thành công hoặc thất bại; xử lý `Idempotency-Key`	`apps/api/src/payments`
CI/CD	GitHub Actions, GHCR	Lint, test, build, đóng gói, deploy, smoke test	`.github/workflows`

Các module backend: `auth`, `products`, `orders` (gồm khuyến mãi và điểm thưởng), `payments`, `prisma`, `health`. Vấn đề xuyên suốt: cấu hình bằng biến môi trường (`.env.example` chỉ có giá trị mẫu); log không ghi mật khẩu, token, secret; `GET /health` cho Docker healthcheck và smoke test; migration chạy trước khi API khởi động; bảo mật bằng bcrypt, JWT có hạn, validate đầu vào, CORS theo `FRONTEND_ORIGIN`.

Đánh đổi. Monorepo cho phép một PR đổi cả hai phía nhưng có hai bộ `package.json`. Modular monolith dễ triển khai và kiểm thử trong một tuần nhưng khó tách dịch vụ về sau. Payment mô phỏng cho phép dựng kịch bản thành công/thất bại nhưng không phản ánh độ trễ và lỗi mạng của dịch vụ ngoài. PostgreSQL có transaction và khóa dòng phù hợp tồn kho đồng thời nhưng cần quản lý migration và sao lưu.

5.2 Quyết định kiến trúc (ADR)

Các ADR toàn văn ở `docs/architecture/adr`; Bảng 15 tóm tắt.

Table 15: Danh sách ADR (đều ở trạng thái đã chốt; ADR-007 chọn ngày 23/10/2026)

ADR	Quyết định	Lý do chính
001	Monorepo `apps/web` và `apps/api`	Một PR đổi cả hai phía, dùng chung CI
002	PostgreSQL 16 và Prisma 7 (đã cân nhắc TypeORM)	Transaction, khóa dòng, migration
003	JWT có hạn, bcrypt, phân quyền CUSTOMER/STAFF; `JWT_SECRET` lấy từ biến môi trường	Đơn giản, dễ kiểm thử
004	Payment là module NestJS; `Idempotency-Key` với ràng buộc duy nhất `(userId, key)` và `requestHash`	Không trừ tiền lặp khi gửi lại
005	Trừ kho bằng `UPDATE ... WHERE stock >= n` trong transaction; nguyên liệu chỉ trừ khi thanh toán thành công	Không âm kho khi đặt và thanh toán đồng thời

ADR	Quyết định	Lý do chính
006	GitHub Actions và GHCR; secret trong GitHub Secrets; bật required checks `api`, `web`	Mã trên GitHub, CI chặn merge
007	Render: web và API chạy image từ GHCR, PostgreSQL của Render; CD gọi Deploy Hook	HTTPS sẵn, gói miễn phí

5.3 Sơ đồ triển khai (Deployment)

Hình 11 vẽ cấu trúc tĩnh của hệ thống theo ký hiệu chuẩn: node là hình hộp 3D có stereotype (`<<device>>`, `<<execution environment>>`, `<<database server>>`, `<<network>>`), artifact nằm trong node chứa nó, đường nối ghi giao thức, và quan hệ `<<deploy>>` từ image trên GHCR sang dịch vụ Render. Trình duyệt truy cập web và API qua Internet bằng HTTPS; API kết nối PostgreSQL của Render bằng Internal Database URL (CSDL không public). Biến môi trường và yêu cầu Public của package GHCR được ghi chú thay vì vẽ thành node. Phần quy trình CI/CD (động) được tách riêng ở Hình 12.

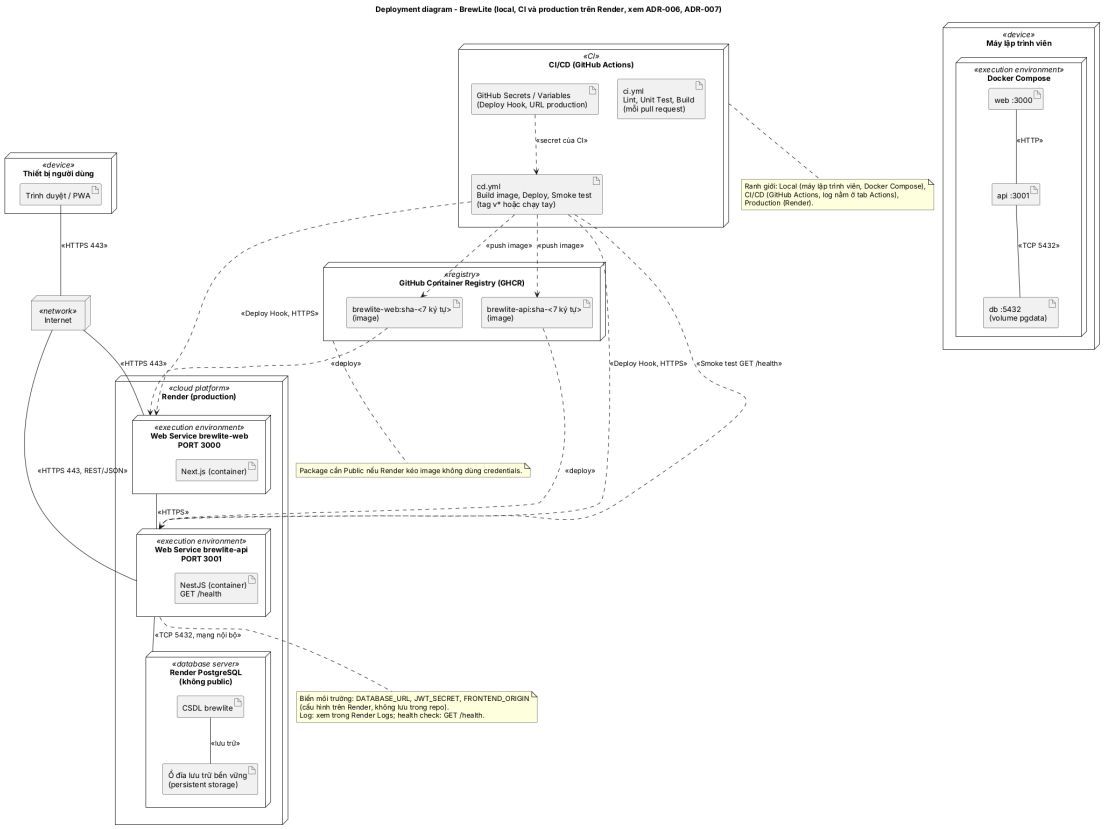


Figure 11: Sơ đồ triển khai (deployment) của BrewLite

Nguồn: nhóm vẽ bằng PlantUML (`docs/architecture/deployment-diagram.puml`)
Nguồn: nhóm vẽ bằng PlantUML (`docs/architecture/cicd-pipeline.puml`); từ `.github/workflows/ci.yml` và `cd.yml`

Giới hạn của gói miễn phí (kiểm tra 23/10/2026): dịch vụ web ngủ khi không dùng (yêu cầu đầu tiên có thể chậm khoảng 50 giây); CSDL hết hạn ngày 22/11/2026 và không có sao lưu.

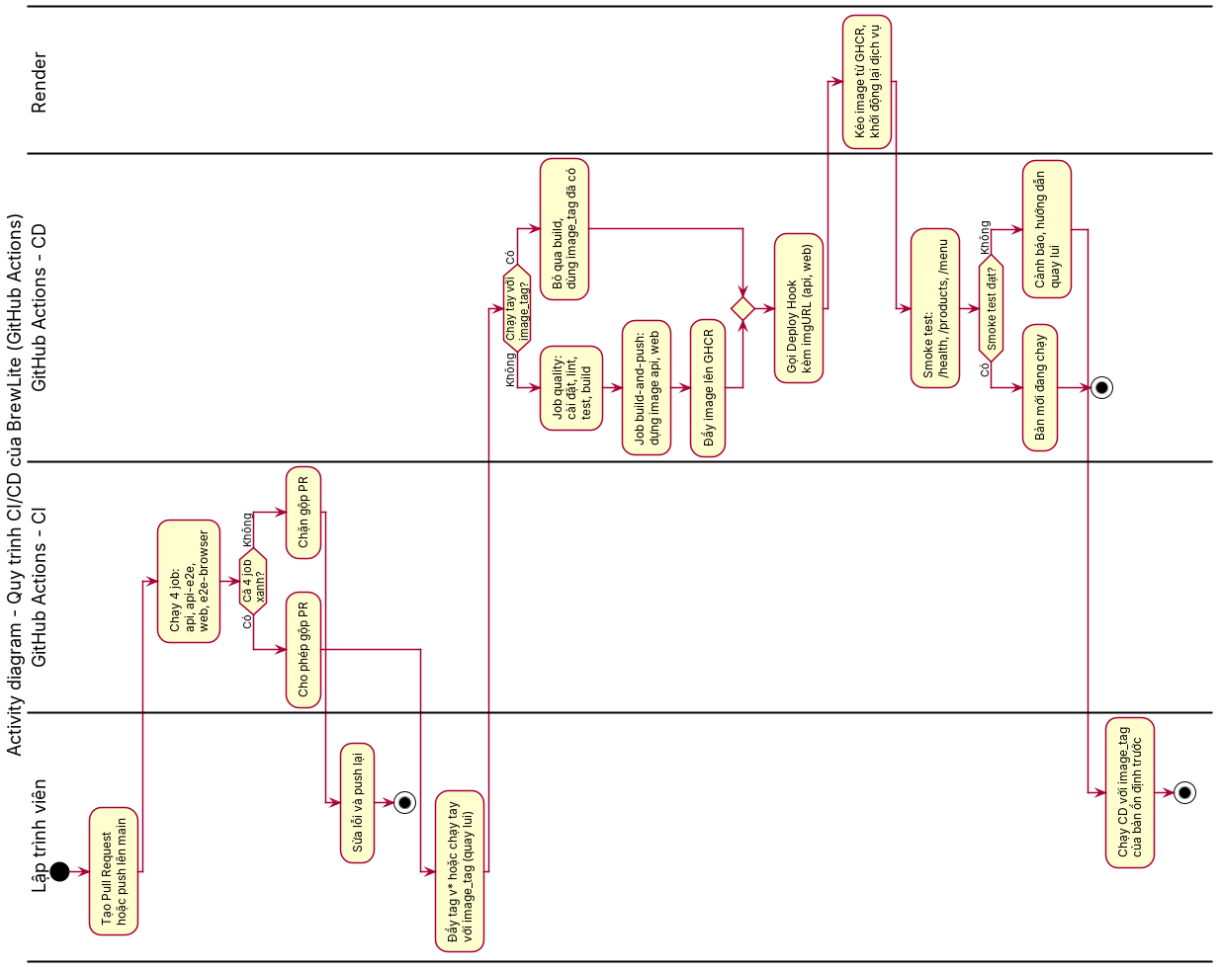


Figure 12: Quy trình CI/CD của BrewLite (activity diagram)

5.4 Cơ sở dữ liệu và API

Lược đồ Prisma gồm 10 thực thể (Hình 13). Khuyến mãi liên kết với đơn qua mã `promoCode`, không dùng khóa ngoại.

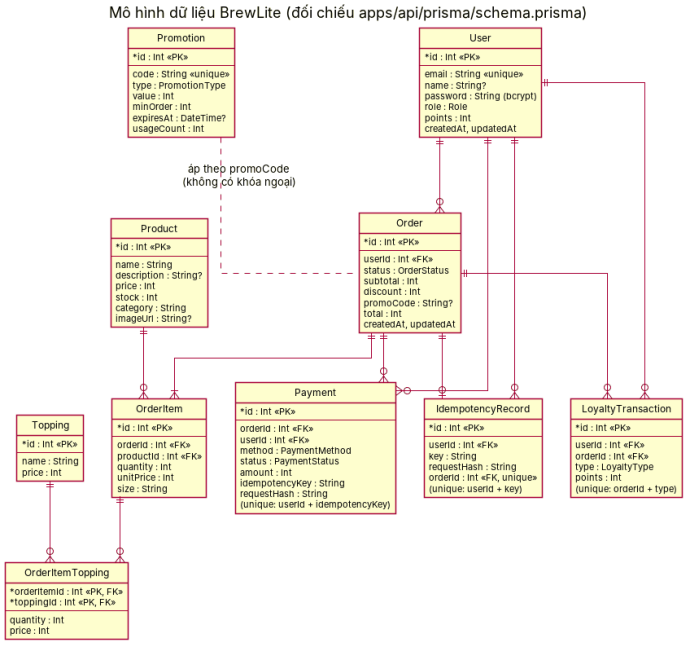


Figure 13: Sơ đồ thực thể – quan hệ theo `schema.prisma`

Nguồn: nhóm vẽ từ `apps/api/prisma/schema.prisma`

Table 16: Các endpoint chính

Phương thức	Đường dẫn	Quyền	Mã lỗi thường gặp
POST	`/auth/register`	Công khai	400, 409 (trùng email)
POST	`/auth/login`	Công khai	401 (sai thông tin)
GET	`/auth/me`	JWT	401
GET	`/products`, `/products/:id`, `/toppings`	Công khai	400 (id sai), 404
POST	`/orders` (kèm `Idempotency-Key`)	JWT	400, 401, 409
GET	`/orders/me`, `/orders/:id`	JWT (chủ đơn hoặc STAFF)	401, 404
GET	`/orders`	STAFF	401, 403
PATCH	`/orders/:id/status`	STAFF	400 (chuyển sai), 403, 409
POST	`/payments` (kèm `Idempotency-Key`)	JWT	400, 401, 404, 409
GET	`/health`	Công khai	–

6 Triển khai và tích hợp liên tục

6.1 Kho mã và quản lý tài liệu

Kho `tthan-prog/BrewLite` theo cấu trúc monorepo: `apps/web` (Next.js), `apps/api` (NestJS, Prisma), `tests/e2e` (Playwright), `docs` (yêu cầu, sơ đồ, kiến trúc, Scrum, bằng chứng, release notes), `.github/workflows` (`ci.yml`, `cd.yml`) và `docker-compose.yml`. Mỗi chức năng một nhánh; thay đổi vào `main` đi qua pull request. Nhánh `main` được bảo vệ: phải qua PR, hai check `api` và `web` phải xanh, cần một approval, không force-push. Tag `assign1-final` (commit `e7a627a`) đánh dấu bộ tài liệu Assignment 1; bản phát hành `v0.1.2` có ghi chú trong `docs/as2/release-notes`.

6.2 Hiện thực các chức năng chính

Luồng menu → size, đường, đá → giỏ → thanh toán → xác nhận → lịch sử đã chạy trên production (23/10/2026, tài khoản `customer@brewlite.com`), kể cả thanh toán thất bại và thử lại. Các ảnh dưới đây là bằng chứng thu thập trong `docs/as2/evidence`.

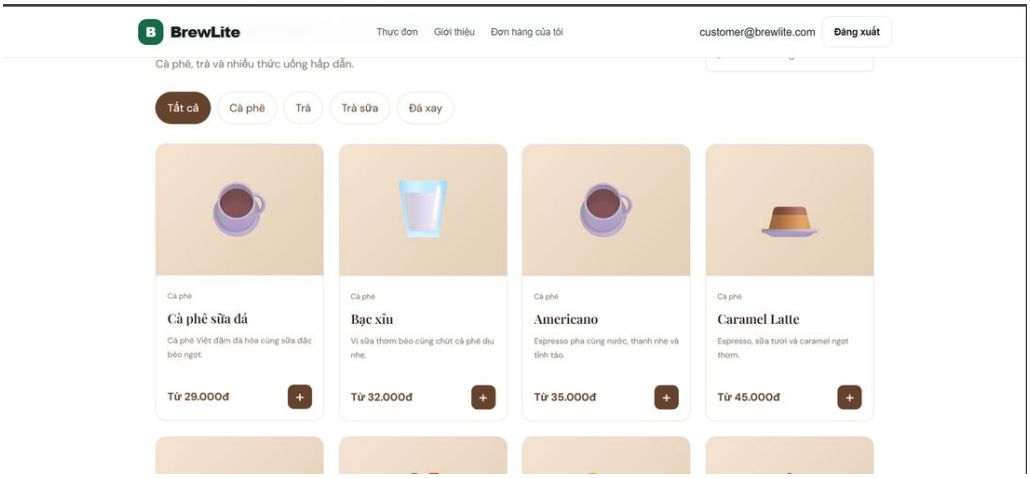


Figure 14: Menu tải từ API trên production

Nguồn: `docs/as2/evidence/01–11`, nhóm chụp ngày 23/10/2026

Kết nối an toàn và endpoint health của production ở Hình 22 và Hình 23.

6.3 CI

`ci.yml` chạy cho mọi PR với bốn job: `api` (cài đặt, lint, unit test, build), `api-e2e` (migration, seed, test tích hợp trên PostgreSQL thật), `web` (lint, build) và `e2e-browser` (Playwright). Lần chạy gần nhất trên `main` (run #60) xanh ở cả bốn job.

Minh chứng CI chặn merge (Task 4.4). Nhóm tạo PR thử nghiệm #30, cố ý đổi kỳ vọng một test trong `order-state.spec.ts` (đơn PAID không được nhảy thẳng sang COMPLETED). Kết quả: test lỗi ở máy (Hình 24), CI run #13 đỏ ở job `api` (Hình 25), PR bị chặn merge vì check `api` là Required (Hình 26); sau khi sửa, run #14 xanh (Hình 27). PR #30 được đóng, không merge.



Figure 15: Chọn size, mức đường, mức đá

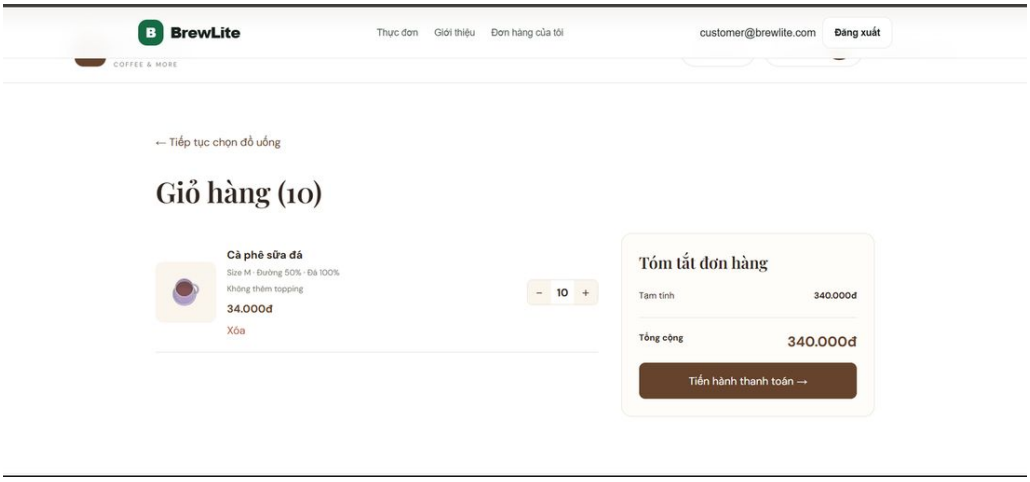


Figure 16: Giỏ hàng: 10 ly, 34.000 đồng/ly, tạm tính 340.000 đồng

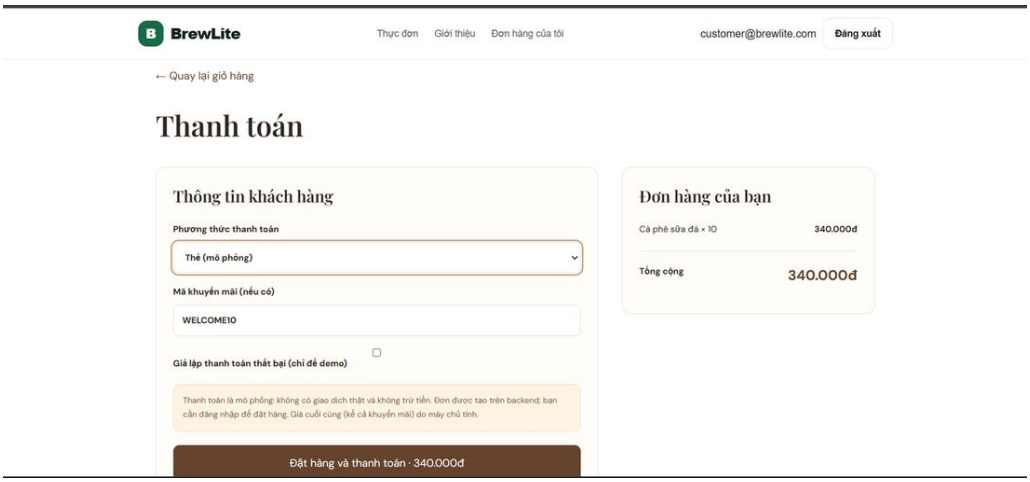


Figure 17: Thanh toán bằng thẻ mô phỏng, mã `WELCOME10`

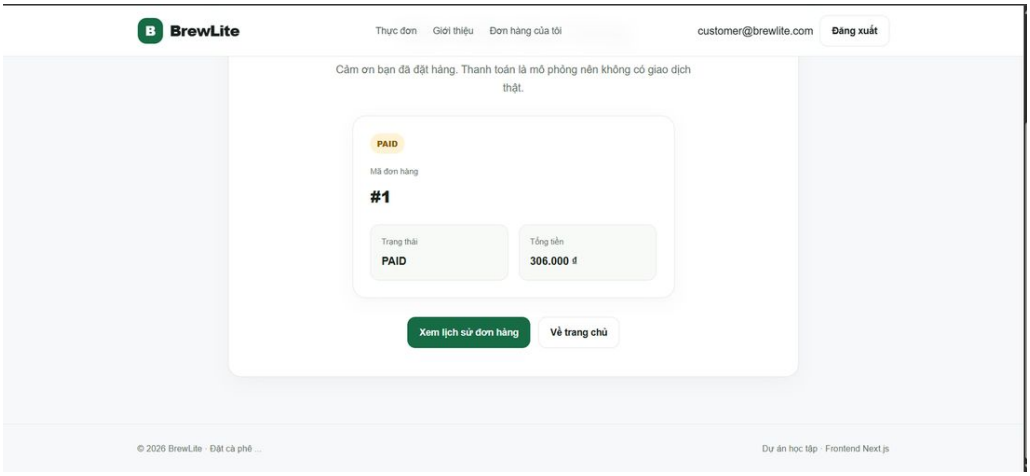


Figure 18: Đơn #1 PAID, 306.000 đồng (340.000 đồng giảm 10%)

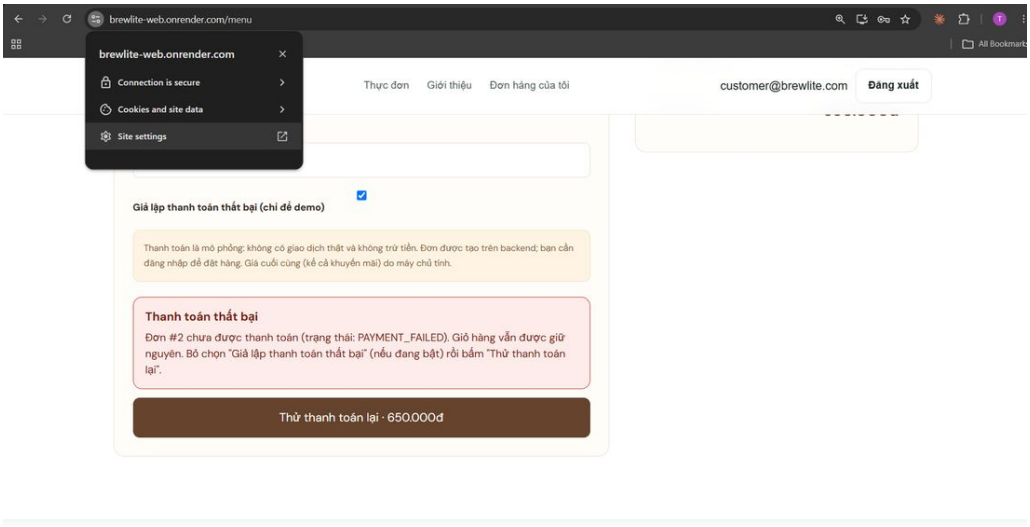


Figure 19: Giả lập thanh toán thất bại: đơn #2 PAYMENT_FAILED, có nút “Thử thanh toán lại”

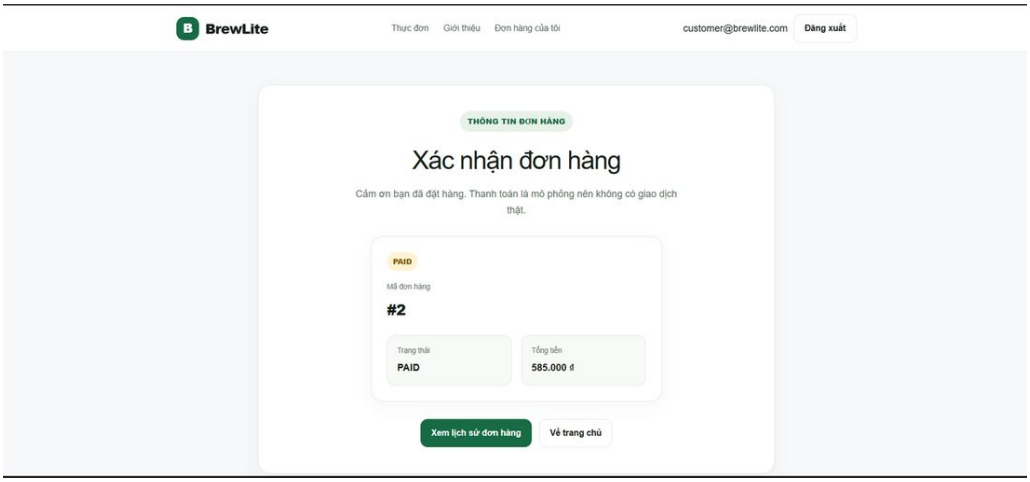


Figure 20: Thử lại thành công: đơn #2 PAID, 585.000 đồng

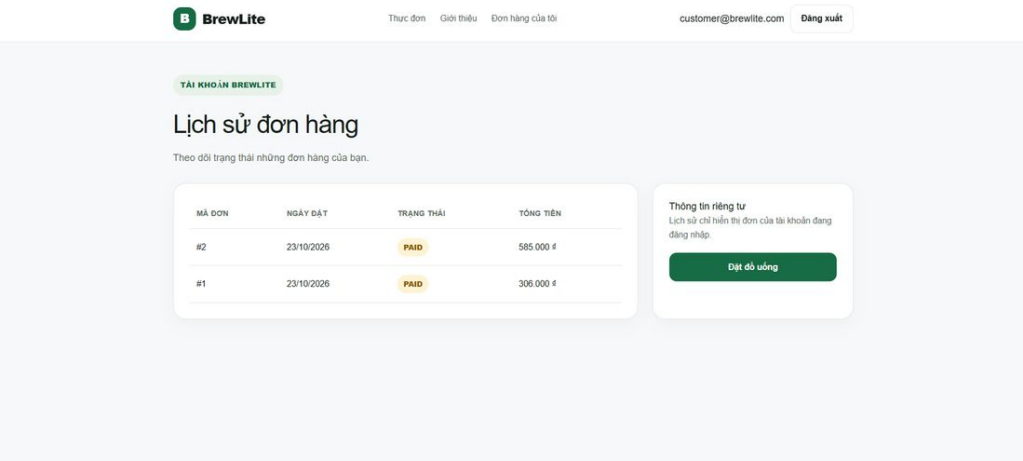


Figure 21: Lịch sử có đúng hai đơn #1 và #2, không tạo đơn trùng khi thử lại

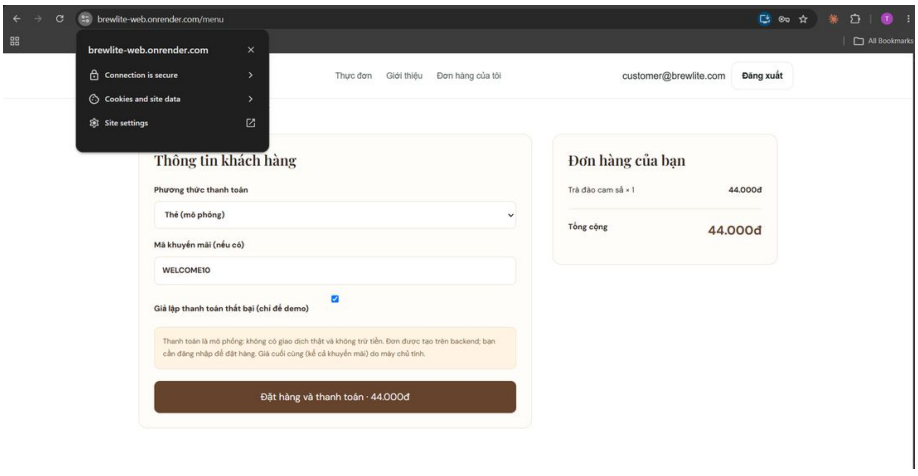


Figure 22: Trình duyệt báo kết nối an toàn (HTTPS)

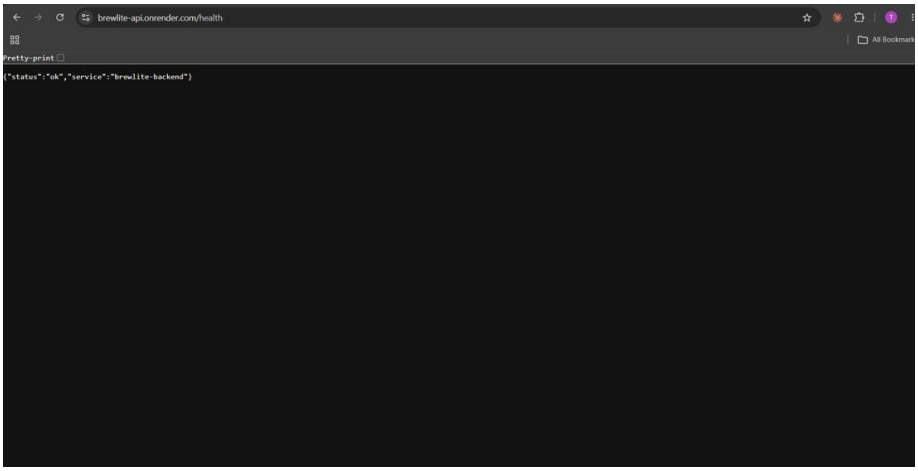


Figure 23: `/health` trả `{"status":"ok","service":"brewlite-backend"}`

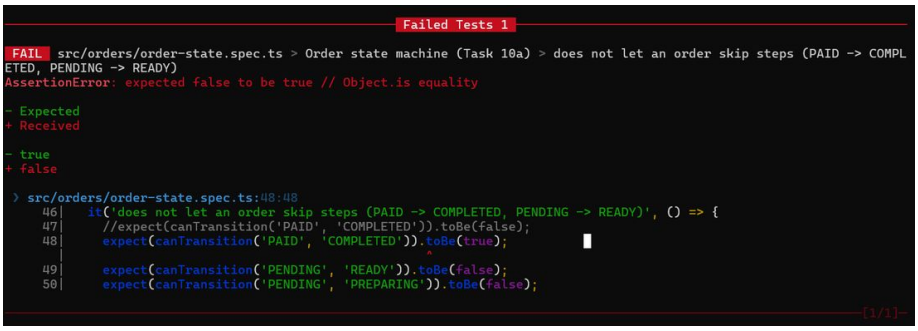


Figure 24: Test lỗi khi chạy ở máy: `expected false to be true`

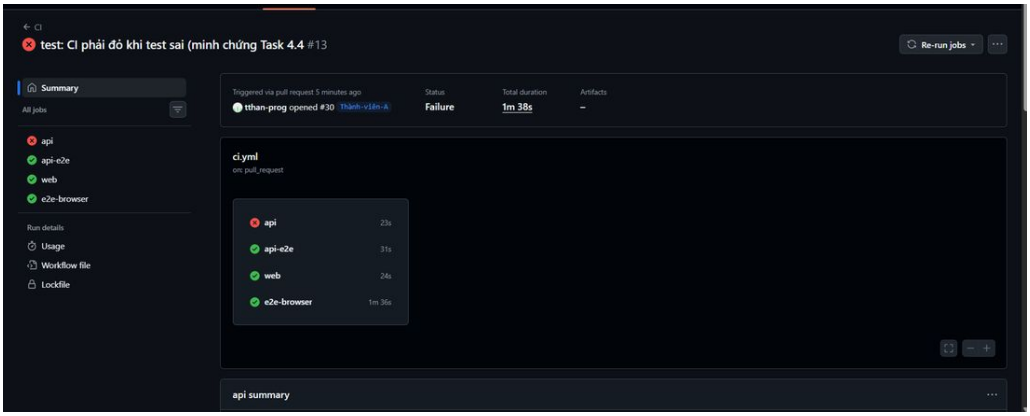


Figure 25: CI run #13 đỏ: job `api` lỗi, các job còn lại xanh

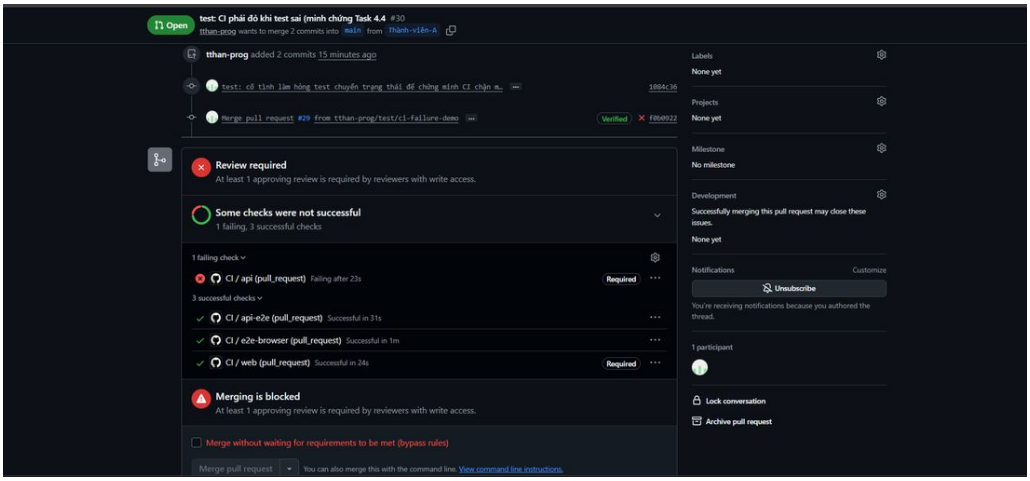


Figure 26: PR #30: check `api` (Required) đỏ, "Merging is blocked"

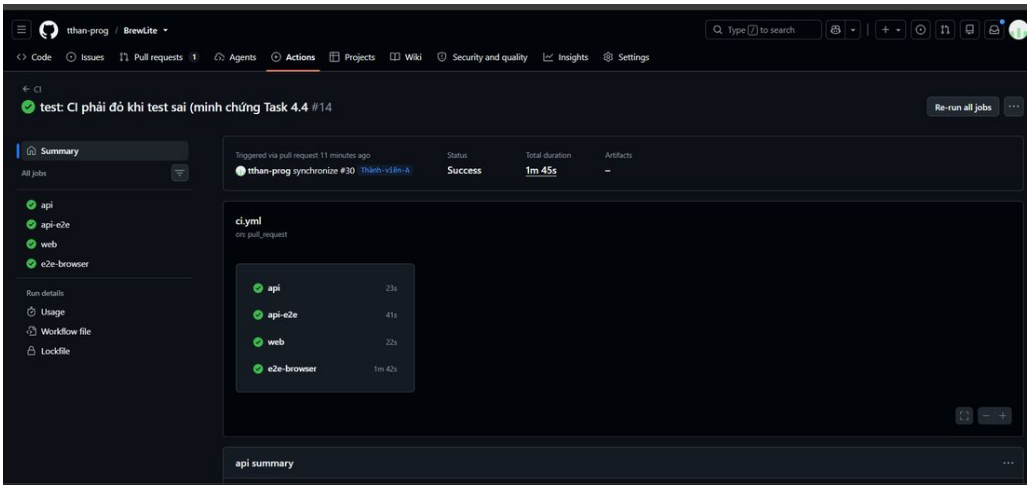


Figure 27: CI run #14 xanh sau khi sửa

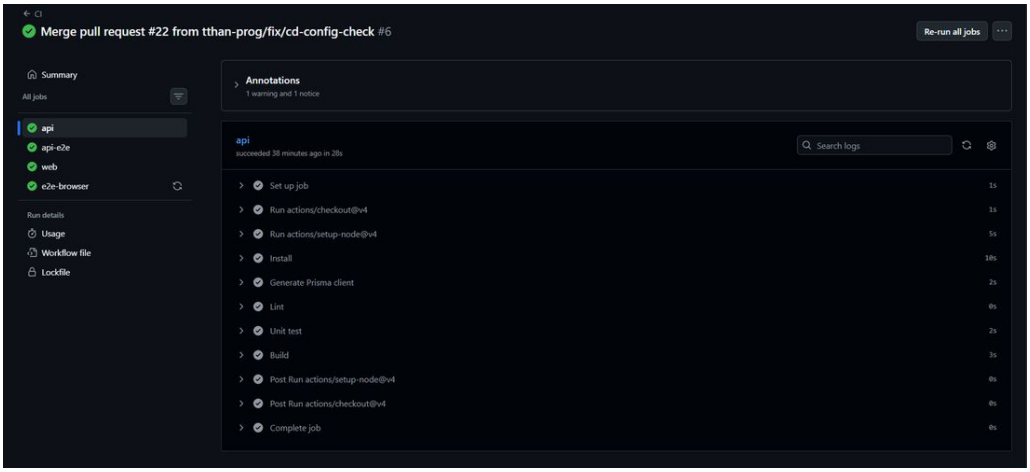


Figure 28: CI xanh 4/4 job (api, api-e2e, web, e2e-browser) sau khi merge PR #22

6.4 CD, di trú dữ liệu và quay lui

`cd.yml` chạy khi gắn tag `v*` hoặc chạy tay (`workflow_dispatch`, tham số `image_tag`) theo ba bước: (1) *quality*: lint và test; (2) *build*: dựng image Docker, đẩy lên GHCR với tag `sha-...`; (3) *deploy*: gọi Deploy Hook của Render cho API và web kèm `imgURL`, rồi smoke test gọi `/health`. Secret nằm trong GitHub Secrets; migration chạy trước khi API khởi động. CD lần đầu (tag `v0.1.0`) đỏ vì thiếu cấu hình URL và hook; sau khi bổ sung, CD xanh từ `v0.1.1` và `v0.1.2`.

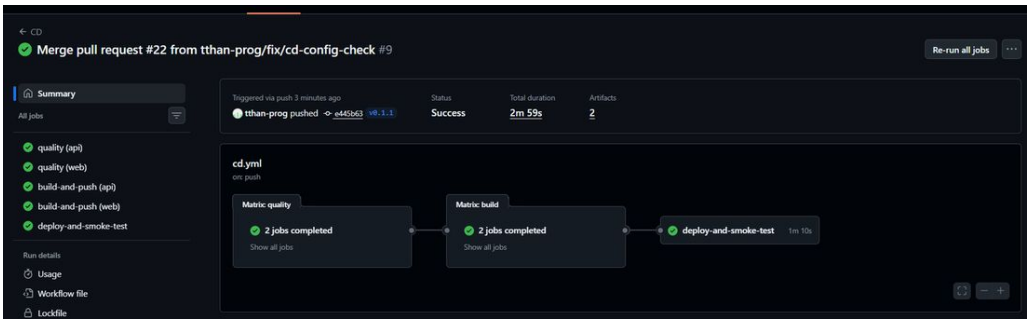


Figure 29: CD từ tag `v0.1.1`: các bước quality, build-and-push, deploy-and-smoke-test đều xanh

Quay lui có minh chứng (23/10/2026). Nhóm chạy lại workflow CD với `image_tag` của bản ổn định trước đó (`sha-e445b63`, tức `v0.1.1`; CD #11): bước build bị bỏ qua, chỉ deploy lại image cũ và chạy smoke test; giao diện quay về bản cũ (không có khung lỗi đỏ). Sau đó triển khai lại bản mới `sha-7b1ea96` (CD #12), khung lỗi đỏ xuất hiện trở lại, chứng tỏ hai lần deploy khác nhau.

7 Kiểm thử

7.1 Chiến lược kiểm thử

Bốn mức kiểm thử, mỗi mức gắn với một nhóm yêu cầu:

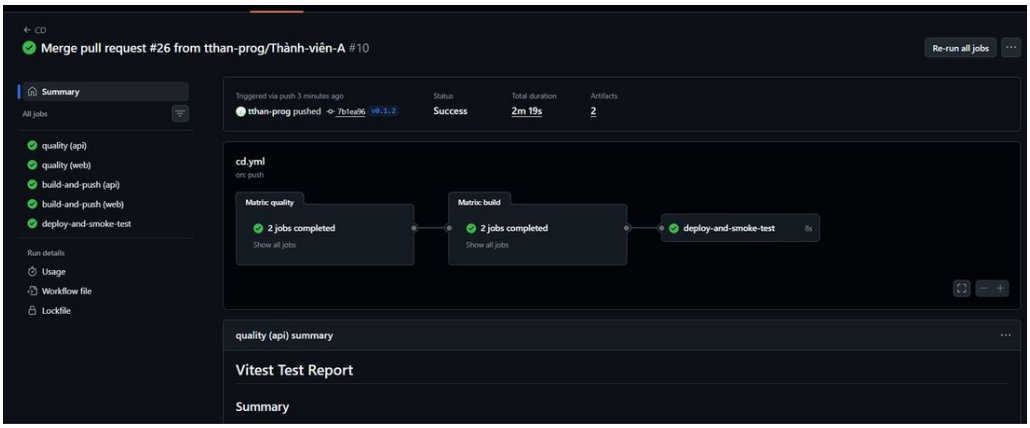


Figure 30: CD từ tag `v0.1.2` (commit `7b1ea96`) xanh

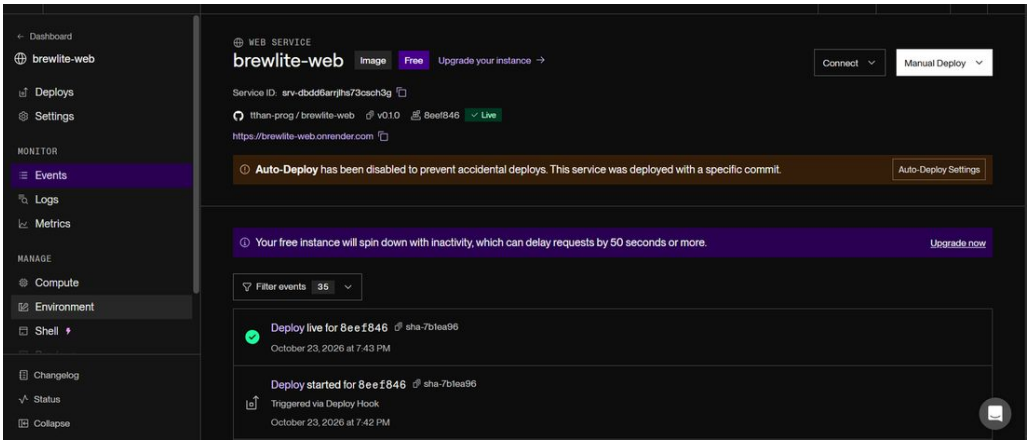


Figure 31: Render Events: deploy image `sha-7b1ea96` qua Deploy Hook

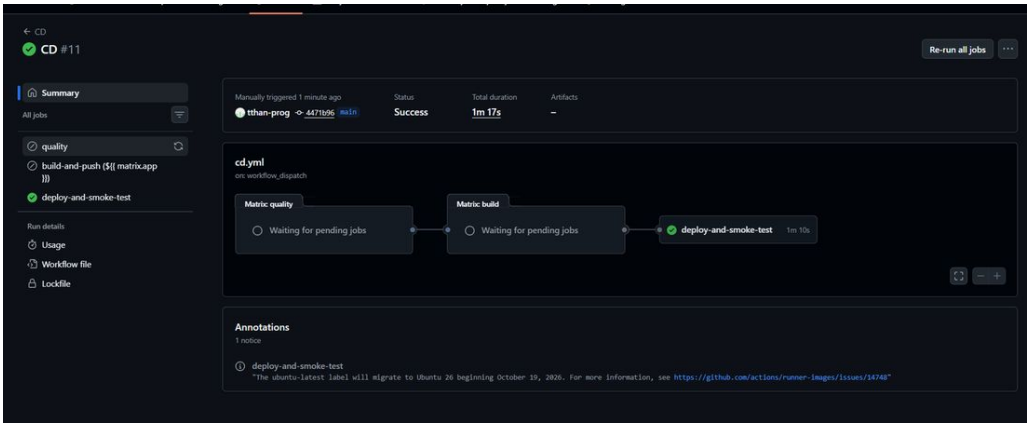


Figure 32: CD #11: bỏ qua build, deploy lại image cũ

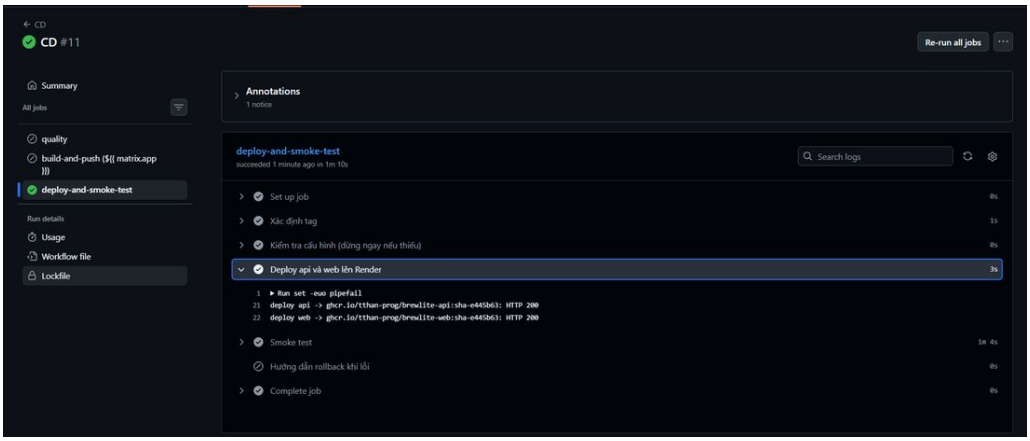


Figure 33: Log deploy `sha-e445b63`

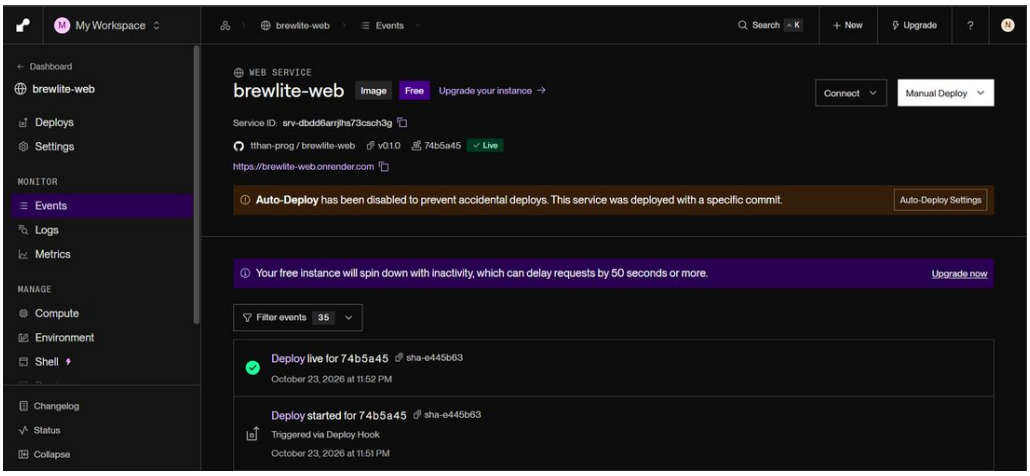


Figure 34: Render Events của lần quay lui

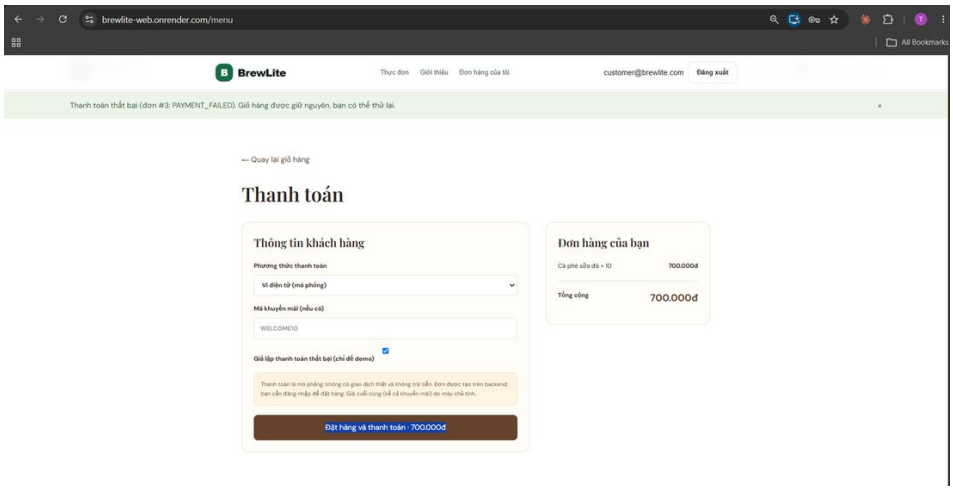


Figure 35: Giao diện bản cũ sau quay lui

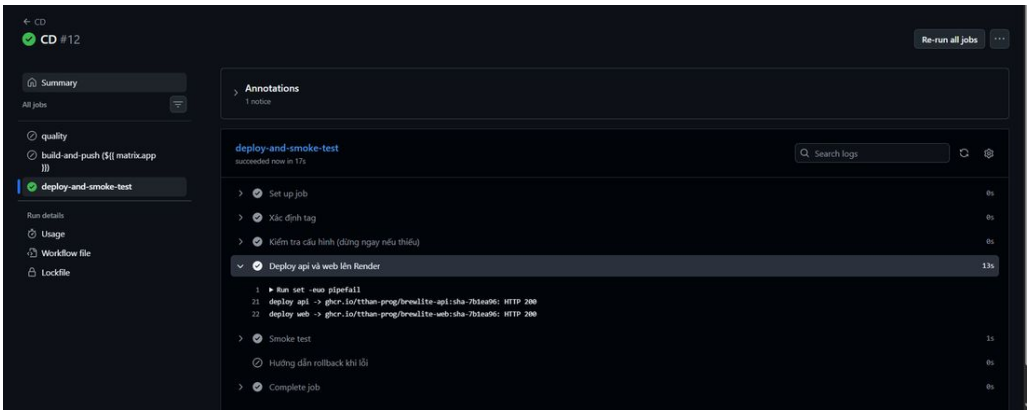


Figure 36: CD #12: triển khai lại `sha-7b1ea96`

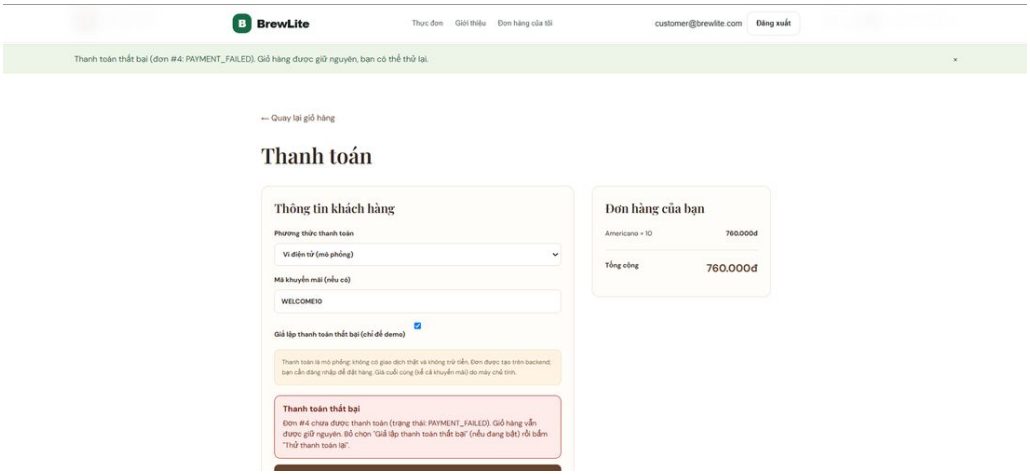


Figure 37: Giao diện bản mới

- **Unit** (Vitest): logic thuần như máy trạng thái đơn và tính điểm thưởng.
- **Tích hợp** (Supertest, PostgreSQL thật, job `api-e2e`): API xác thực, đơn, thanh toán, idempotency, tồn kho, khuyến mãi.
- **E2E trình duyệt** (Playwright, job `e2e-browser`): giỏ hàng, viewport di động, ba trạng thái menu, manifest PWA.
- **Smoke** (trong `cd.yml`): gọi `/health` sau mỗi lần deploy.

Số test lấy từ lần chạy gần nhất trên nhánh `main` (commit `2cae795`); kết quả đạt/không đạt lấy từ CI.

Table 17: Số lượng test theo mức

Mức	Công cụ	Số test	Vị trí
Unit	Vitest	36	`apps/api/src/**/*.spec.ts`
Tích hợp	Supertest	81	`apps/api/test/`: `app.e2e-spec.ts` (2), `assign1.e2e-spec.ts` (30), `cors.e2e-spec.ts` (2), `qr-ingredients.e2e-spec.ts` (31), `realism.e2e-spec.ts` (16)
E2E trình duyệt	Playwright	10	`tests/e2e/brewlite-flow.spec.ts`

7.2 Test case

Bảng 18 liệt kê các test case chính, lấy tên từ mã nguồn kiểm thử và gắn với FR/NFR. Bảy test Playwright đầu tiên là E2E-T01a/b, T02, T03a/b/c và T04, ba test còn lại thuộc các nhóm bổ sung của tập 10 test.

Table 18: Test case chính và yêu cầu liên quan

Mã test	Mục tiêu (kỳ vọng)	Yêu cầu	Mức
INT-T01	`GET /products` trả danh sách có id, name, price, imageUrl	FR-01	Tích hợp
INT-T02	`GET /products/:id`: có → 200, không có → 404, id sai → 400	FR-03	Tích hợp
INT-T03	Hash bcrypt; trùng email 409, sai mật khẩu 401; đăng nhập đúng trả JWT dùng được	FR-05, NFR-02	Tích hợp
INT-T04	`POST /orders` thiếu token → 401	FR-06	Tích hợp
INT-T05	Giá theo size; validate thiếu items, size sai, số lượng $\leq 0 \rightarrow 400$; `userId` trong body bị bỏ qua	FR-07, NFR-04	Tích hợp
INT-T06	Thanh toán thành công → PAID; lỗi → PAYMENT_FAILED, thử lại được rồi PAID	FR-08	Tích hợp
INT-T07	Hai lần cùng khóa chỉ một đơn; 10 yêu cầu song song cùng khóa; không thanh toán lần hai	FR-09	Tích hợp
API-T01, INT-T08	Mọi cặp trạng thái; chuyển sai bị chặn (400)	FR-10	Unit, tích hợp

Mã test	Mục tiêu (kỳ vọng)	Yêu cầu	Mức
INT-T09	STAFF đi hết PAID → PREPARING → READY → COMPLETED; CUSTOMER gọi PATCH nhận 403	FR-11, NFR-02	Tích hợp
INT-T10	`/orders/me` chỉ trả đơn của mình; `/orders/:id`: chủ đơn 200, người khác 404	FR-12	Tích hợp
INT-T11	20 người đặt song song, kho 5 → đúng 5 đơn; đơn lỗi giữa chừng rollback; hủy đơn chỉ hoàn phần đã trừ	FR-13	Tích hợp
INT-T12	Mã `WELCOME10` giảm 10% khi đủ điều kiện	FR-14	Tích hợp
API-T02, INT-T13	1 điểm cho mỗi 10.000 đồng; PAID cộng điểm; hủy đơn đã PAID trừ lại (EARN/REVERSAL)	FR-15	Unit, tích hợp
E2E-T01a/b	Thêm vào giỏ và tính tổng tiền	FR-04	E2E
E2E-T02	Viewport 390 px không cuộn ngang	NFR-05	E2E
E2E-T03a/b/c	Menu đang tải, rỗng, lỗi và thử lại	FR-02, NFR-06	E2E
E2E-T04	Manifest và biểu tượng PWA	NFR-05	E2E

7.3 Kết quả kiểm thử

Tự động. Lần chạy gần nhất trên `main` (commit `2cae795`) cho 36 unit, 81 tích hợp và 10 E2E đều đạt, CI xanh ở cả bốn job. Ba test bắt buộc của Task 10 đều đạt: chặn chuyển trạng thái sai, cùng `Idempotency-Key` chỉ tạo một đơn, 20 yêu cầu song song khi kho còn 5 thì đúng 5 thành công.

Thủ công. Thành viên D chạy trực tiếp trên backend Docker (`localhost:3001`) sáu trường hợp, tất cả đạt (Bảng 19).

Table 19: Kiểm thử chức năng thủ công

Trường hợp	Kết quả
`GET /products`	12 sản phẩm
`GET /products/1`	Cà phê sữa đá, 29.000 đồng
`GET /products/999999`	HTTP 404
`POST /orders` không có JWT	HTTP 401
`POST /orders` có JWT, thiếu `Idempotency-Key`	HTTP 400
`POST /orders` hợp lệ	Đơn #6, PENDING, 34.000 đồng

Hiệu năng (NFR-01). Đo thủ công ngày 24/10/2026 bằng Docker Compose cục bộ với 12 sản phẩm: lần chậm thứ 19 trong 20 yêu cầu `GET /products` mất 19,7 ms, đạt ngưỡng 500 ms. Chưa có test tự động. Trên production (Render gói miễn phí, Singapore), thời gian tới byte đầu tiên đo từ Việt Nam khoảng 0,43–0,78 giây; yêu cầu đầu tiên sau thời gian không dùng có thể chậm khoảng 50 giây.

Hạn chế đã biết. Chưa có test tự động cho NFR-01 và NFR-03 (secret mới dò sơ bộ bằng tìm theo mẫu ngày 24/10, chưa rà thủ công đầy đủ); NFR-06 chưa có test kiểm không lộ stack trace; NFR-07 chưa có người ngoài nhóm chạy thử theo README; chưa có độ phủ (coverage) được đo.

8 Bàn giao và đánh giá nhìn lại

8.1 Sản phẩm bàn giao

Table 20: Danh mục bàn giao

Sản phẩm	Vị trí / liên kết
Kho mã nguồn	https://github.com/tthan-prog/BrewLite
Web production (HTTPS)	https://brewlite-web.onrender.com
API và health	https://brewlite-api.onrender.com/health
Bản phát hành	Tag v0.1.2; ghi chú trong docs/as2/release-notes
SRS, use case, truy vết	docs/requirements, docs/traceability-matrix.md
Sơ đồ UML (nguồn và ảnh)	docs/diagrams, docs/architecture
Kiến trúc và ADR	docs/architecture/architecture.md, adr/001–007
CI/CD và quay lui	.github/workflows/ci.yml, cd.yml; docs/deployment.md; README mục 15
Test tự động	36 unit, 81 tích hợp, 10 E2E
Bằng chứng	docs/as2/evidence (ảnh 01–25)
Biên bản Sprint	docs/as2/scrum/minutes/a2-*
Video demo end-to-end	https://drive.google.com/file/d/1uxrJSc9OkNkD-68vCTKXsPb0DcUudtat/view?usp=sharing (Giản Thiên Lực thực hiện)

CSDL production không public (kết nối nội bộ); không có secret trong repository (.env bị .gitignore chặn, giá trị thật nằm trong GitHub Secrets và Render).

8.2 Sprint Review / Demo

Increment demo được là phiên bản v0.1.2 trên production. Bảng 21 tóm tắt Review Sprint 2 (24/10, cập nhật đến 26/10). Sprint Goal phần lớn đạt: có URL production HTTPS, CI/CD chạy, quay lui đã chứng minh; video demo được hoàn tất sau buổi Review.

Table 21: Kết quả Sprint Review Sprint 2

Hạng mục	Demo	Đạt DoD	Bằng chứng
5.1 Hoàn thiện MVP	Có	Có	Ảnh 01–11
5.2 Tích hợp, kiểm thử	Có	Có	CI 4 job, 10 E2E
5.3 CD và quay lui	Có	Có	Ảnh 12–14, 20–25
5.4 Production, release, README	Có	Gần đạt	Release v0.1.2
5.4 Video demo	Có (sau Review)	Chờ nghiệm thu	Liên kết ở Bảng 20
5.4 Review, Retrospective	Có	Có	Biên bản

8.3 Retrospective

Điều làm tốt, nên giữ: quay lui bằng `image_tag` chạy được và có minh chứng; smoke test và `/health` bắt lỗi sau triển khai; bằng chứng ảnh được đặt tên có thứ tự; mọi thay đổi vào `main` đi qua PR và CI; CI chặn merge có bằng chứng đỏ và xanh.

Điều cần cải thiện: CD lần đầu đỏ vì thiếu cấu hình URL và hook; Render gói miễn phí làm web ngủ và CSDL hết hạn 22/11/2026, không có sao lưu; nghiệm thu chéo chưa ghi trên PR (cột Reviews trống); một lần tạo PR nhầm nhánh đích; một lần gõ sai mã khuyến mãi (`WELCOM10`) khi thử làm tưởng lỗi giao diện.

Table 22: Hành động sau Sprint

Hành động	Người làm	Kiểm tra vào
Quay video demo end-to-end	Giản Thiên Lực	Đã xong, liên kết ở Bảng 20
Mở web production một lần trước giờ chấm để tránh chờ khởi động	Nguyễn Lê Nhựt Thắng	Trước buổi chấm
Ghi kết quả nghiệm thu chéo vào PR (comment hoặc Approve)	Người nghiệm thu theo bảng Review	Trước buổi chấm
Sao lưu hoặc làm mới CSDL trước 22/11 nếu còn dùng	Nguyễn Lê Nhựt Thắng	Sau khi nộp

9 Ma trận truy vết

Ma trận nối yêu cầu → use case → thiết kế → mã nguồn → test. Bản đầy đủ và được cập nhật ở `docs/traceability-matrix.md` (23/10/2026); cột "Mã nguồn" và "Test" trỏ tới mã và test có thật trong repo. Ô ghi "–" nghĩa là không áp dụng hoặc chưa làm, không điền trước. Mã test: API-Txx là unit, INT-Txx là tích hợp, E2E-Txx là Playwright. Mọi yêu cầu mức Must đều có thiết kế, mã và test, trừ các điểm ghi ở cột Trạng thái.

Table 23: Ma trận truy vết yêu cầu

Yêu cầu	Ưu tiên	Use case	Thiết kế	Mã nguồn	Test	Trạng thái
FR-01	Must	US-01	Activity, Class	`GET /products`	INT-T01	Đạt
FR-02	Must	US-01	Activity	Trang menu	E2E-T03a/b/c	Đạt
FR-03	Must	US-02	Class	`GET /products/:id`	INT-T02	Đạt
FR-04	Must	US-03	Activity	Giỏ hàng (web)	E2E-T01a/b	Đạt
FR-05	Must	US-04	Class (User)	`POST /auth/*`	INT-T03	Đạt
FR-06	Must	UC-01 (E1)	Activity, Sequence	`JwtAuthGuard`	INT-T04	Đạt
FR-07	Must	UC-01	Sequence, Class	`POST /orders`	INT-T05	Đạt
FR-08	Must	UC-01	Sequence, Class	`POST /payments`	INT-T06	Đạt
FR-09	Must	UC-01 (A2)	Sequence, ADR-004	`Idempotency-Key`	INT-T07	Đạt
FR-10	Must	UC-01, UC-02	Class (OrderStatus)	`order-state.ts`	API-T01, INT-T08	Đạt
FR-11	Should	UC-02	Class (Role)	`PATCH /orders/:id/status`	INT-T09	Đạt
FR-12	Must	US-05	Class	`GET /orders/me`	INT-T10	Đạt
FR-13	Must	UC-01 (E3)	Sequence, ADR-005	Cập nhật có điều kiện	INT-T11	Đạt
FR-14	Should	UC-01 (A1)	Class (Promotion)	`promoCode`	INT-T12	Đạt
FR-15	Should	UC-02	Class (LoyaltyTransaction)	`pointsForTotal`	API-T02, INT-T13	Đạt
FR-16	Could	–	–	–	–	Ngoài MVP
FR-17	Won't	–	–	–	–	Không làm
NFR-01	Should	–	Architecture	`GET /products`	Đo thủ công	Đạt, chưa tự động

Yêu cầu	Ưu tiên	Use case	Thiết kế	Mã nguồn	Test	Trạng thái
NFR-02	Must	–	ADR-003	bcrypt, `JwtAuthGuard`	INT-T03, T09	Đạt
NFR-03	Must	–	ADR-006	`.gitignore`, GitHub Secrets	Dò sơ bộ	Chưa rà đủ
NFR-04	Must	–	–	class-validator	INT-T05	Đạt
NFR-05	Must	–	Kiến trúc (web)	CSS, `manifest.ts`	E2E-T02, T04	Đạt, chưa service worker
NFR-06	Should	–	–	Thông báo tiếng Việt	E2E-T03c	Một phần
NFR-07	Must	–	Deployment	`docker-compose.yml`	Chạy 24/10	Chưa có người ngoài nhóm thử
NFR-08	Must	–	ADR-006	`ci.yml`, `cd.yml`	CI #13/#14, CD	Đạt
NFR-09	Must	–	Deployment	`GET /health`	Smoke test	Đạt
NFR-10	Must	–	ADR-002	`prisma/migrations`, seed	Job `api-e2e`	Đạt
NFR-11	Should	–	ADR-006	Job `api`, `web`	CI	Đạt

10 Kết luận

10.1 Kết quả đạt được

Nhóm hoàn thành luồng end-to-end từ menu đến lịch sử đơn và chạy trên production qua HTTPS, gồm cả thanh toán thất bại và thử lại không tạo đơn trùng. Yêu cầu được đặc tả thành 17 FR và 11 NFR có tiêu chí chấp nhận; ba sơ đồ UML, sơ đồ trạng thái, kiến trúc, ERD, deployment và bảy ADR được rà theo mã nguồn. CI có bốn job và đã chứng minh chặn merge khi test lỗi; CD triển khai từ tag, có smoke test và quay lui đã chạy thật. Có 36 unit, 81 tích hợp và 10 E2E test; ma trận truy vết nối mọi yêu cầu Must với thiết kế, mã và test.

10.2 Hạn chế

- Nghiệm thu chéo chưa có Approve hoặc bình luận trên GitHub; PR cột Reviews còn trống.
- NFR-01 chỉ đo thủ công một lần; NFR-03 mới dò sơ bộ; chưa đo độ phủ test.
- Render gói miễn phí: web ngủ khi không dùng, CSDL hết hạn 22/11/2026 và không có sao lưu.
- Thanh toán chỉ mô phỏng; PWA mới ở mức manifest; chưa có giao diện nhân viên và quản trị.
- Phân bố commit giữa các thành viên chưa đều.

10.3 Hướng phát triển

Bổ sung service worker để PWA dùng được offline; giao diện barista và quản trị sản phẩm, khuyến mãi; kết nối cổng thanh toán thật; lưu mức đường và đá vào đơn; tự động hóa đo hiệu năng và quét bí mật trong CI; chuyển sang gói hạ tầng có sao lưu.

A Bảng đối chiếu yêu cầu đề bài với báo cáo

Các hạng mục lấy theo cách đánh số Task của Assignment 2 trong tài liệu của nhóm (`docs/as2/scrum/minutes/a2-*`).

Hạng mục của đề bài	Mục trong báo cáo	Trạng thái
1.1 Bối cảnh, stakeholder, phạm vi	mục 3.1, mục 3.2, mục 3.3	Đã đáp ứng
1.2 SRS (FR, NFR, ưu tiên, tiêu chí)	mục 3.4, mục 3.5	Đã đáp ứng
1.3 Use case và đặc tả	mục 3.6	Đã đáp ứng
2 Activity, Sequence, Class	mục 4.1, mục 4.2, mục 4.3	Đã đáp ứng
3.1 Kiến trúc và quyết định thiết kế	mục 5.1, mục 5.2	Đã đáp ứng
3.2 Deployment diagram	mục 5.3	Đã đáp ứng
4.1 Repository, quản lý tài liệu	mục 6.1, mục 2.4	Đã đáp ứng
4.3 MVP Sprint 1	mục 6.2, Bảng 6	Đã đáp ứng
4.4 CI, một lần đỏ và một lần xanh	mục 6.3	Đã đáp ứng
5.1 Hoàn thiện luồng MVP	mục 6.2	Đã đáp ứng
5.2 Tích hợp và kiểm thử	mục 7	Đáp ứng một phần (chưa đo coverage)
5.3 CD và quay lui	mục 6.4	Đã đáp ứng
5.4 URL, release, README, bàn giao	mục 8.1	Đã đáp ứng
5.4 Sprint Review, Retrospective	mục 8.2, mục 8.3	Đã đáp ứng
Ma trận truy vết	mục 9	Đã đáp ứng

B Tư liệu bổ sung

B.1 Cấu trúc kho mã

Listing 1: Cấu trúc thư mục chính của repository

```
BrewLite/
 apps/api/    NestJS, Prisma (schema, migrations, seed), test
 apps/web/    Next.js (App Router)
 tests/e2e/   Playwright
 docs/        requirements, diagrams, architecture (ADR),
              traceability-matrix, deployment, as1, as2, chung
 .github/workflows/ ci.yml, cd.yml
 docker-compose.yml
```

B.2 Liên kết

Kho mã: https://github.com/tthan-prog/BrewLite. Web production: https://brewlite-web.onrender.com. API: https://brewlite-api.onrender.com/health. Video demo: https://drive.google.com/file/d/1uxrJSc9OkNkD-68vCTKXsPb0DcUudtat/view?usp=sharing.

References

[1] Ken Schwaber, Jeff Sutherland, "**The Scrum Guide**", *link: https://scrumguides.org/scrum-guide.html*, 2020, lần truy cập cuối: 26/10/2026.

[2] Khoa Công nghệ thông tin, Trường Đại học Sài Gòn, "**BrewLite – Assignment 2: Requirements to Deployment (VER 1.0)**", *môn Công nghệ phần mềm, học kỳ I năm học 2026–2027*.